
User Manual

for MPC560XB SPI Driver

Document Number: UM13SPIASR4.0R1.0.1
Rev 1.2





Contents

Section number	Title	Page
Chapter 1		
Revision History		
Chapter 2		
Introduction		
2.1	Supported Derivatives.....	11
2.2	Overview.....	12
2.3	About this Manual.....	12
2.4	Acronyms and Definitions.....	12
2.5	Reference List.....	13
Chapter 3		
Driver		
3.1	Requirements.....	15
3.2	Driver Design Summary.....	15
3.3	Deviation from Requirements.....	16
3.4	Function Definitions.....	17
3.4.1	Function Spi_AsyncTransmit.....	18
3.4.2	Function Spi_Cancel.....	18
3.4.3	Function Spi_DeInit.....	19
3.4.4	Function Spi_GetAsyncStatus.....	19
3.4.5	Function Spi_GetHWUnitStatus.....	20
3.4.6	Function Spi_GetJobResult.....	21
3.4.7	Function Spi_GetSequenceResult.....	21
3.4.8	Function Spi_GetStatus.....	22
3.4.9	Function Spi_GetVersionInfo.....	23
3.4.10	Function Spi_Init.....	23
3.4.11	Function Spi_MainFunction_Handling.....	24
3.4.12	Function Spi_ReadIB.....	24
3.4.13	Function Spi_SetAsyncMode.....	25

Section number	Title	Page
3.4.14	Function Spi_SetupEB.....	26
3.4.15	Function Spi_SyncTransmit.....	27
3.4.16	Function Spi_WriteIB.....	27
3.4.17	Function Spi_SetHWUnitAsyncMode.....	28
3.4.18	Function Spi_SetClockMode.....	29
3.4.19	Function Index.....	29
3.5	Enum Definitions.....	30
3.5.1	Enumeration Spi_DualClockMode.....	30
3.5.2	Enumeration Spi_JobResultType.....	31
3.5.3	Enumeration Spi_SeqResultType.....	31
3.5.4	Enumeration Spi_StatusType.....	31
3.5.5	Enumeration Spi_AsyncModeType.....	32
3.5.6	Enumeration Spi_ModeType.....	32
3.6	Structure Definitions.....	32
3.6.1	Structure Spi_HWUnitConfig.....	32
3.6.2	Structure Spi_ChannelConfig.....	33
3.6.3	Structure Spi_JobConfig.....	34
3.6.4	Structure Spi_SequenceConfig.....	34
3.7	Define Definitions.....	35
3.7.1	Define SPI_ASYNCTRANSMIT_ID.....	35
3.7.2	Define SPI_CANCEL_ID.....	35
3.7.3	Define SPI_DEINIT_ID.....	35
3.7.4	Define SPI_GETHWUNITSTATUS_ID.....	35
3.7.5	Define SPI_GETJOBRESULT_ID.....	36
3.7.6	Define SPI_GETSEQUENCERESULT_ID.....	36
3.7.7	Define SPI_GETSTATUS_ID.....	36
3.7.8	Define SPI_GETVERSIONINFO_ID.....	36
3.7.9	Define SPI_INIT_ID.....	36
3.7.10	Define SPI_MAINFUNCTION_HANDLING_ID.....	36

Section number	Title	Page
3.7.11	Define SPI_MODULE_ID.....	36
3.7.12	Define SPI_READIB_ID.....	37
3.7.13	Define SPI_SETASYNCMODE_ID.....	37
3.7.14	Define SPI_SETCLOCKMODE_ID.....	37
3.7.15	Define SPI_SETHWUNITASYNCMODE_ID.....	37
3.7.16	Define SPI_SETUPEB_ID.....	37
3.7.17	Define SPI_SYNCTRANSMIT_ID.....	37
3.7.18	Define SPI_VENDOR_ID.....	38
3.7.19	Define SPI_WRITEIB_ID.....	38
3.7.20	Define SPI_E_ALREADY_INITIALIZED.....	38
3.7.21	Define SPI_E_CONFIG_OUT_OF_RANGE.....	38
3.7.22	Define SPI_E_JOB_EMPTY.....	38
3.7.23	Define SPI_E_PARAM_CHANNEL.....	38
3.7.24	Define SPI_E_PARAM_EB_UNIT.....	38
3.7.25	Define SPI_E_PARAM_JOB.....	39
3.7.26	Define SPI_E_PARAM_LENGTH.....	39
3.7.27	Define SPI_E_PARAM_SEQ.....	39
3.7.28	Define SPI_E_PARAM_UNIT.....	39
3.7.29	Define SPI_E_SEQ_EMPTY.....	39
3.7.30	Define SPI_E_SEQ_IN_PROCESS.....	39
3.7.31	Define SPI_E_SEQ_PENDING.....	40
3.7.32	Define SPI_E_UNINIT.....	40
3.8	Symbolic Names DISCLAIMER.....	40
3.9	Configuration Parameters.....	40
3.9.1	Pre-Compile Parameters.....	41
3.9.1.1	SpiCancelApi.....	42
3.9.1.2	SpiChannelBuffersAllowed.....	43
3.9.1.3	SpiDevErrorDetect	43
3.9.1.4	SpiHWStatusAPI.....	43

Section number	Title	Page
3.9.1.5	SpiInterruptibleSeqAllowed.....	44
3.9.1.6	SpiLevelDelivered.....	44
3.9.1.7	SpiVersionInfoApi.....	45
3.9.1.8	SpiClockRef.....	45
3.9.1.9	SpiGlobalDmaEnable.....	45
3.9.1.10	SpiSyncTransmitTimeout.....	46
3.9.1.11	SpiOptimizeOneJobSequences.....	46
3.9.1.12	SpiOptimizedChannelsNumber.....	47
3.9.1.13	SpiOptimizedSeqNumber.....	47
3.9.1.14	SpiEnableHWUnitAsyncMode	48
3.9.1.15	SpiEnableDualClockMode.....	48
3.9.1.16	SpiAlternateClockRef.....	49
3.9.1.17	SpiAllowBigSizeCollections.....	49
3.9.1.18	SpiForceDataType.....	49
3.9.2	Post-Build parameters.....	50
3.9.2.1	SpiPhyUnitMapping.....	53
3.9.2.2	SpiPhyUnitSync.....	54
3.9.2.3	SpiPhyUnitMode.....	54
3.9.2.4	SpiPhyUnitAsyncMethod.....	55
3.9.2.5	SpiPhyTxDmaChannel.....	55
3.9.2.6	SpiPhyTxDmaChannelAux.....	56
3.9.2.7	SpiPhyRxDmaChannel.....	56
3.9.2.8	SpiMaxChannel.....	57
3.9.2.9	SpiChannelId.....	58
3.9.2.10	SpiChannelType.....	58
3.9.2.11	SpiDataWidth.....	58
3.9.2.12	SpiDefaultData.....	59
3.9.2.13	SpiEbMaxlength.....	59
3.9.2.14	SpiIbNBuffers.....	60

Section number	Title	Page
3.9.2.15	SpiTransferStart.....	60
3.9.2.16	SpiBaudRate.....	61
3.9.2.17	SpiCs.....	62
3.9.2.18	SpiCsPolarity.....	62
3.9.2.19	SpiDataShiftEdge.....	63
3.9.2.20	SpiEnableCs.....	63
3.9.2.21	SpiShiftClockIdleLevel.....	64
3.9.2.22	SpiTimeClk2Cs.....	65
3.9.2.23	SpiTimeCs2Clk.....	65
3.9.2.24	SpiTimeCs2Cs.....	66
3.9.2.25	SpiCsContinuous.....	66
3.9.2.26	SpiMaxJob.....	67
3.9.2.27	SpiHwUnit.....	67
3.9.2.28	SpiJobId.....	68
3.9.2.29	SpiJobPriority.....	68
3.9.2.30	SpiChannelAssignment.....	69
3.9.2.31	SpiDeviceAssignment.....	70
3.9.2.32	SpiMaxSequence.....	70
3.9.2.33	SpiInterruptibleSequence.....	71
3.9.2.34	SpiSeqEndNotification.....	71
3.9.2.35	SpiSequenceId.....	72
3.9.2.36	SpiJobAssignment.....	72
3.9.2.37	SpiJobEndNotification.....	73
3.9.2.38	SpiJobstartNotification.....	73

Chapter 4

How to use the same channel in multiple post build Configurations



Chapter 1

Revision History

Table 1-1. Document Change History

Date	Version	Changed by	Change description
17-Feb-2012	1.0	Subramanya M Naik	Initial Release
14-May-2013	1.1	Manojraj L	Update for RTM 1.0.0 release
15-May-2014	1.2	Hoang Ngoc Hien	Update for RTM 1.0.1 release



Chapter 2

Introduction

This User's Manual describes Freescale Semiconductor AUTOSAR Serial Peripheral interface (SPI) driver.

AUTOSAR SPI driver configuration parameters and deviations from the specification are described in SPI Driver chapter. AUTOSAR SPI driver requirements and APIs are described in the AUTOSAR SPI driver software specification document (See [Table Reference List](#) - #1).

2.1 Supported Derivatives

The software described in this document is intended to be used with the following microcontroller devices of Freescale Semiconductor:

Table 2-1. Supported Derivatives

Freescale Semiconductor	mpc5601dxlh_lqfp64 mpc5601dxll_lqfp100 mpc5602dxlh_lqfp64 mpc5602dxll_lqfp100 mpc5602bxlh_lqfp64 mpc5602bxll_lqfp100 mpc5602bxlq_lqfp144 mpc5602cxlh_lqfp64 mpc5602cxll_lqfp100 mpc5603bxlh_lqfp64 mpc5603bxll_lqfp100 mpc5603bxlq_lqfp144 mpc5603cxlh_lqfp64 mpc5603cxll_lqfp100 mpc5604bxlh_lqfp64 mpc5604bxll_lqfp100 mpc5604bxlq_lqfp144 mpc5604bxmg_mapbga208 mpc5604cxlh_lqfp64 mpc5604cxll_lqfp100 mpc5605bxll_lqfp100 mpc5605bxlq_lqfp144 mpc5605bxlu_lqfp176 mpc5606bkll_lqfp100
-------------------------	--

Table 2-1. Supported Derivatives

	mpc5606bxlq_lqfp144 mpc5606bxlu_lqfp176 mpc5607bxlu_lqfp176 mpc5607bxmg_mapbga208
--	--

All of the above microcontroller devices are collectively named as MPC560XB.

2.2 Overview

AUTOSAR (Automotive Open System Architecture) is an industry partnership working to establish standards for software interfaces and software modules for automobile electronic control systems.

2.3 About this Manual

This Technical Reference employs the following typographical conventions:

Boldface type: Bold is used for important terms, notes and warnings.

Italic font: Italic typeface is used for code snippets in the text. Note that C language modifiers such "const" or "volatile" are sometimes omitted to improve readability of the presented code.

2.4 Acronyms and Definitions

Table 2-2. Acronyms and Definitions

Term	Definition
API	Application Programming Interface
AUTOSAR	Automotive Open System Architecture
BSMI	Basic Software Make file Interface
CS	Chip Select
DEM	Diagnostic Event Manager
DET	Development Error Tracer
ECU	Electronic Control Unit
FIFO	First In First Out
MIDE	Multi Integrated Development Environment
MCU	Micro Controller Unit

Table continues on the next page...

Table 2-2. Acronyms and Definitions (continued)

Term	Definition
LSB	Least Significant Bit
MSB	Most Significant Bit
RAM	Random Access Memory
SIU	Systems Integration Unit
SPI	Serial Peripheral Interface
SWS	Software Specification
VLE	Variable Length Encoding
XML	Extensible Markup Language
N/A	Not Applicable

2.5 Reference List

Table 2-3. Reference List

Title	Version
AUTOSAR SPI Driver Software Specification Document	V3.2.0
MPC5607B Reference Manual	Rev. 7.2, May 2012
MPC5604B Reference Manual	Rev. 8.2, September 2013
5606B Reference Manual	Rev. 2, May 2014
5602D Reference Manual	Rev. 4.2, Dec 2013
Bolero 1.5M cut 2.2 Errata Sheet (MPC5607B_1M03Y)	Rev. 12Dec2013
Bolero 512k cut 2.3 Errata Sheet (MPC5604B_2M27V)	Rev. 26 FEB 2014
Bolero 1M cut 1.0 Errata Sheet (MPC5606B_0N13E)	Rev. , 12 DEC 2013
Bolero 256k cut 1.1 Errata Sheet (MPC5602D_1M18Y)	Rev. , 12 DEC 2013

Chapter 3

Driver

3.1 Requirements

Requirements for this driver are detailed in the AUTOSAR SPI Driver software specification document. (See Table [Reference List - #1](#)).

3.2 Driver Design Summary

The SPI Handler and Driver provide services for reading from and writing to devices connected via SPI busses. It provides access to SPI communication to several users (e.g., EEPROM, Watchdog, I/O ASICs). It also provides the required mechanism to configure the on-chip SPI peripheral.

This specification describes the API, Mapping to SWS requirements for a monolithic SPI Handler and Driver. This software module includes handling and driving functionalities. Main objectives of this monolithic SPI Driver are to take the best of each microcontroller features and to allow implementation optimization depending on static configuration to fit as much as possible to ECU needs.

The general behavior of the SPI Handler and Driver could be asynchronous or synchronous according to the level of functionality selected.

The specification covers the Handler and Driver functionalities combined in one single module. The SPI handler controls multiple accesses to busses that could be located in the ECU Abstraction layer. The other part is the SPI driver that accesses the microcontroller hardware directly that could be located in the Microcontroller Abstraction layer.

SPI Dual Clock Mode

The SPI Driver allows to be used in Dual Clock Mode. This mode permits to change the clock reference(referred by the field SpiAlternateClockRef) and to keep the basic characteristics of the transmission(like baudrate). This is useful when it wants to be crossed to a low frequency(low power) or higher frequency.

Interrupt request usage

Every interrupt is guarded by #ifdef definitions that specify if the corresponded DSPI is used. If not, the interrupt function is removed. A template of the #ifdef guard is:

```
#if ((SPI_LEVEL_DELIVERED == LEVEL1) || (SPI_LEVEL_DELIVERED == LEVEL2))
```

```
#if (<DSPI_ENABLED> == STD_ON)
```

```
<ISR_function_name()>
```

```
#endif
```

```
#endif
```

Description of the symbolic names

When the plugin is generated, symbolic names of the sequences, jobs and channels are created by define macros. The templates of the defines are:

- for sequences: #define SpiConf_SpiSequence_<SpiSequenceName>
((Spi_SequenceType)<SpiSequenceID>)
- for jobs: #define SpiConf_SpiJob_<SpiJobName> ((Spi_JobType)<SpiJobID>)
- for channels: #define SpiConf_SpiChannel_<SpiChannelName>
((Spi_ChannelType)<SpiChannelID>)

Name is the name of the container and the ID is configurable by the user

3.3 Deviation from Requirements

Table 3.1 Deviations Status Column Description identifies the AUTOSAR requirements that are not fully implemented, implemented differently, or out of scope for the SPI driver. Table [Table 3-2](#) provides Status column description.

Table 3-1. Deviations Status Column Description

Term	Definition
N/A	Not available

Table continues on the next page...

Table 3-1. Deviations Status Column Description (continued)

Term	Definition
N/T	Not testable
N/S	Out of scope
N/I	Not implemented
N/F	Not fully implemented

Below table identifies the AUTOSAR requirements that are not fully implemented, implemented differently, or out of scope for the SPI driver.

Table 3-2. SPI Deviations Table

Requirement	Status	Description	Notes
SPI270	N/S	In case call end notification function and rescheduling are fully done by software, the order between these shall be first scheduling and then the call of end notification function executed.	Job and sequences notifications are performed before the scheduling of the next job (contrary to the recommendation given by SPI270) . In this way, calls like Spi_SetupIB() or Spi_WriteIB() can be targeted on the next schedulable jobs, before the starting of the job transfer.
SPI238_Conf	N/S	If SpiHwUnitSynchronous is set to "Synchronous" for a job, the associated bus defined by SpiHwUnit behaves same as prearranged bus. It means that all requirements valid for prearranged bus will be valid also for the bus assigned to this job.	The SpiHwUnitSynchronous parameter is defined at Job level in Autosar. It should be defined at HwUnit level. This parameter is added for AutoSAR compatibility purpose and this parameter is not used. The parameter SpiPhyUnitSync located in SpiPhyUnit tab is used for the same purpose.
SPI239_Conf	N/S	SpiCsSelection {SPI_CS_SELECTION} When the Chip select handling is enabled (see SpiEnableCs), then this parameter specifies if the chip select is handled automatically by Peripheral HW engine or via general purpose IO by Spi driver.	If user selects parameter CS_VIA_GPIO, the user has to use SpiJobStartNotification' & 'SpiJobEndNotification' to toggle the CS (chip select pin) using DIO drivers .
SPI195	N/S	SPI Handler/driver shall be able to detect the error SPI_E_HARDWARE_ERROR when an hardware error occur during asynchronous transmit.	SPI_E_HARDWARE_ERROR is not supported for Spi_Async_Transmit() Function. It is supported for Spi_Sync_Transmit() function. To implement this requirement a timer should be set to that estimated value and if the timer expires, then it can be assumed that a hardware error occurred. This would add a dependency of Gpt.

3.4 Function Definitions

APIs of all functions supported by the driver are as per AUTOSAR SPI Driver software specification Version 4.0 Rev0003.

3.4.1 Function Spi_AsyncTransmit

This function triggers the asynchronous transmission for the given sequence.

Prototype: Std_ReturnType Spi_AsyncTransmit(Spi_SequenceType Sequence);

Table 3-3. Spi_AsyncTransmit Arguments

Type	Name	Direction	Description
Spi_SequenceType	Sequence	input	Sequence ID

Return: Std_ReturnType

Table 3-4. Spi_AsyncTransmit Returns

Value	Description
E_OK	Transmission command has been accepted
E_NOT_OK	Transmission command has not been accepted

This function triggers the asynchronous transmission for the given sequence.

- Service ID: 0x03
- Sync/Async: Asynchronous
- Reentrancy: Reentrant

pre:: The driver needs to be initialized before calling Spi_AsyncTransmit() otherwise, the function Spi_AsyncTransmit() shall raise the development error if SPI_DEV_ERROR_DETECT is STD_ON. Pre-compile parameter SPI_LEVEL_DELIVERED shall be LEVEL1 or LEVEL2.

3.4.2 Function Spi_Cancel

This function is used to request the cancelation of the given sequence.

Prototype: void Spi_Cancel(Spi_SequenceType Sequence);

Table 3-5. Spi_Cancel Arguments

Type	Name	Direction	Description
Spi_SequenceType	Sequence	input	Sequence ID

This function is used to request the cancelation of the given sequence.

- Service ID: 0x0c
- Sync/Async: Asynchronous
- Reentrancy: Reentrant

pre:: The driver needs to be initialized before calling `Spi_Cancel()` otherwise, the function `Spi_Cancel()` shall raise the development error if `SPI_DEV_ERROR_DETECT` is `STD_ON`. Pre-compile parameter `SPI_CANCEL_API` shall be `STD_ON`

post:: The SPI Handler/Driver is not responsible on external devices damages or undefined state due to cancelling a sequence transmission.

3.4.3 Function Spi_DeInit

This function de-initializes the SPI driver.

Prototype: `Std_ReturnType Spi_DeInit(void);`

Return: `Std_ReturnType`

Table 3-6. Spi_DeInit Returns

Value	Description
E_OK	de-initialisation command has been accepted
E_NOT_OK	de-initialisation command has not been accepted

This function de-initializes the SPI driver using the pre-established configurations

- Service ID: 0x01
- Sync/Async: Synchronous
- Reentrancy: Non-Reentrant

pre:: The driver needs to be initialized before calling `Spi_DeInit()` otherwise, the function `Spi_DeInit()` shall raise the development error if `SPI_DEV_ERROR_DETECT` is `STD_ON`.

3.4.4 Function Spi_GetAsyncStatus

This function returns the status of the SPI driver related to async HW Units.

Prototype: `Spi_StatusType Spi_GetAsyncStatus(void);`

Return: `Spi_StatusType`

Table 3-7. Spi_GetAsyncStatus Returns

Value	Description
SPI_UNINIT	The driver is un-initialized
SPI_IDLE	The driver has no pending transfers
SPI_BUSY	The driver is busy

Return SPI_BUSY if at least one async HW unit is busy.

Note

Violates Spi_c_REF_1 Violates MISRA 2004 Advisory Rule 19.1: include statements in a file should only be preceded by other preprocessor directives or comments. Violates Spi_c_REF_2 Violates MISRA 2004 Required Rule 19.15: Precautions shall be taken in order to prevent the contents of a header file being included twice.

pre:: Pre-compile parameter SPI_LEVEL_DELIVERED shall be LEVEL2.

3.4.5 Function Spi_GetHWUnitStatus

This function is used to request the status of a specific SPI peripheral unit.

Prototype: Spi_StatusType Spi_GetHWUnitStatus(Spi_HWUnitType HWUnit);

Table 3-8. Spi_GetHWUnitStatus Arguments

Type	Name	Direction	Description
Spi_HWUnitType	HWUnit	input	The HW peripheral for which we need the status

Return: Spi_StatusType

Table 3-9. Spi_GetHWUnitStatus Returns

Value	Description
SPI_UNINIT	The peripheral is un-initialized
SPI_IDLE	The peripheral is in idle state
SPI_BUSY	The peripheral is busy

This function is used to request the status of a specific SPI peripheral unit.

- Service ID: 0x0b
- Sync/Async: Synchronous
- Reentrancy: Reentrant

pre:: The driver needs to be initialized before calling `Spi_GetHWUnitStatus()` otherwise, the function `Spi_GetHWUnitStatus()` shall raise the development error if `SPI_DEV_ERROR_DETECT` is `STD_ON`. `SPI_HW_STATUS_API == STD_ON`

3.4.6 Function Spi_GetJobResult

This function is used to request the status of a specific job.

Prototype: `Spi_JobResultType Spi_GetJobResult(Spi_JobType Job);`

Table 3-10. Spi_GetJobResult Arguments

Type	Name	Direction	Description
<code>Spi_JobType</code>	Job	input	Job ID

Return: `Spi_JobResultType`

Table 3-11. Spi_GetJobResult Returns

Value	Description
<code>SPI_JOB_OK</code>	The job ended successfully
<code>SPI_JOB_PENDING</code>	The job is pending
<code>SPI_JOB_FAILED</code>	The job has failed
<code>SPI_JOB_QUEUED</code>	An asynchronous transmit Job has been accepted, while actual transmission for this Job has not started yet.

This function is used to request the status of a specific job.

- Service ID: 0x07
- Sync/Async: Synchronous
- Reentrancy: Reentrant

pre:: The driver needs to be initialized before calling `Spi_GetJobResult()` otherwise, the function `Spi_GetJobResult()` shall raise the development error if `SPI_DEV_ERROR_DETECT` is `STD_ON`.

3.4.7 Function Spi_GetSequenceResult

This function is used to request the status of a specific sequence.

Prototype: `Spi_SeqResultType Spi_GetSequenceResult(Spi_SequenceType Sequence);`

Table 3-12. Spi_GetSequenceResult Arguments

Type	Name	Direction	Description
Spi_SequenceType	Sequence	input	Sequence ID

Return: Spi_SeqResultType

Table 3-13. Spi_GetSequenceResult Returns

Value	Description
SPI_SEQ_OK	The sequence ended successfully
SPI_SEQ_PENDING	The sequence is pending
SPI_SEQ_FAILED	The sequence has failed

This function is used to request the status of a specific sequence.

- Service ID: 0x08
- Sync/Async: Synchronous
- Reentrancy: Reentrant

pre:: The driver needs to be initialized before calling `Spi_GetSequenceResult()` otherwise, the function `Spi_GetSequenceResult()` shall raise the development error if `SPI_DEV_ERROR_DETECT` is `STD_ON`.

3.4.8 Function Spi_GetStatus

This function returns the status of the SPI driver.

Prototype: `Spi_StatusType Spi_GetStatus(void);`

Return: Spi_StatusType

Table 3-14. Spi_GetStatus Returns

Value	Description
SPI_UNINIT	The driver is un-initialized
SPI_IDLE	The driver has no pending transfers
SPI_BUSY	The driver is busy

This function returns the status of the SPI driver.

- Service ID: 0x06
- Sync/Async: Synchronous
- Reentrancy: Reentrant

pre:: The driver needs to be initialized before calling `Spi_GetStatus()` otherwise, the function `Spi_GetStatus()` shall raise the development error if `SPI_DEV_ERROR_DETECT` is `STD_ON`.

3.4.9 Function Spi_GetVersionInfo

This function returns the version information for the SPI driver.

Prototype: `void Spi_GetVersionInfo(Std_VersionInfoType *VersionInfo);`

Table 3-15. Spi_GetVersionInfo Arguments

Type	Name	Direction	Description
Std_VersionInfoType *	VersionInfo	input, output	Pointer to where to store the version information of this module.

This function returns the version information for the SPI driver.

Note

Violates `Spi_c_REF_1` Violates MISRA 2004 Advisory Rule 19.1: include statements in a file should only be preceded by other preprocessor directives or comments.

- Service ID: 0x09
- Sync/Async: Synchronous
- Reentrancy: Non-Reentrant

pre:: Pre-compile parameter `SPI_VERSION_INFO_API` shall be `STD_ON`.

Note

Violates `Spi_h_REF_3` Violates MISRA 2004 Advisory Rule 11.4: Precautions shall be taken in order to prevent the contents of a header file being included twice.

3.4.10 Function Spi_Init

This function initializes the SPI driver.

Prototype: `void Spi_Init(const Spi_ConfigType *ConfigPtr);`

Table 3-16. Spi_Init Arguments

Type	Name	Direction	Description
const Spi_ConfigType *	ConfigPtr	input	Specifies the pointer to the configuration set

This function initializes the SPI driver using the pre-established configurations

- Service ID: 0x00
- Sync/Async: Synchronous
- Reentrancy: Non-Reentrant

Note

Violates Spi_c_REF_3 Violates MISRA 2004 Advisory Rule 11.4:
A cast should not be performed between a pointer to object type
and a different pointer to object type.

3.4.11 Function Spi_MainFunction_Handling

This function shall asynchronously poll SPI interrupts and call ISR if appropriate.

Prototype: void Spi_MainFunction_Handling(void);

This function shall asynchronously poll SPI interrupts and call ISR if appropriate.

- Service ID: 0x10

pre.: Pre-compile parameter SPI_LEVEL_DELIVERED shall be LEVEL1 or LEVEL2.

3.4.12 Function Spi_ReadIB

This function reads the data from the buffer of a channel and puts at the memory location.

Prototype: Std_ReturnType Spi_ReadIB(Spi_ChannelType Channel, Spi_DataType *DataBufferPtr);

Table 3-17. Spi_ReadIB Arguments

Type	Name	Direction	Description
Spi_ChannelType	Channel	input	Channel ID
Spi_DataType *	DataBufferPtr	input, output	Pointer to the memory location that will be written with the data in the internal buffer

Return: Std_ReturnType

Table 3-18. Spi_ReadIB Returns

Value	Description
E_OK	read command has been accepted
E_NOT_OK	read command has not been accepted

This function reads the data from the buffer of a specific channel and puts at the specified memory location.

- Service ID: 0x04
- Sync/Async: Synchronous
- Reentrancy: Reentrant

pre:: The driver needs to be initialized before calling `Spi_ReadIB()` otherwise, the function `Spi_ReadIB()` shall raise the development error if `SPI_DEV_ERROR_DETECT` is `STD_ON`. Pre-compile parameter `SPI_CHANNEL_BUFFERS_ALLOWED` shall be `USAGE0` or `USAGE2`.

3.4.13 Function Spi_SetAsyncMode

This function specifies the asynchronous mode for the SPI busses handled asynchronously.

Prototype: `Std_ReturnType Spi_SetAsyncMode(Spi_AsyncModeType AsyncMode);`

Table 3-19. Spi_SetAsyncMode Arguments

Type	Name	Direction	Description
<code>Spi_AsyncModeType</code>	<code>AsyncMode</code>	input	This parameter specifies the asynchronous operating mode (<code>SPI_POLLING_MODE</code> or <code>SPI_INTERRUPT_MODE</code>)

Return: `Std_ReturnType`

Table 3-20. Spi_SetAsyncMode Returns

Value	Description
E_OK	The command ended successfully
E_NOT_OK	The command has failed

This function specifies the asynchronous mode for the SPI busses handled asynchronously.

- Service ID: 0x0d

- Sync/Async: Synchronous
- Reentrancy: Non-Reentrant

pre:: The driver needs to be initialized before calling `Spi_SetAsyncMode()` otherwise, the function `Spi_SetAsyncMode()` shall raise the development error if `SPI_DEV_ERROR_DETECT` is `STD_ON`. Pre-compile parameter `SPI_LEVEL_DELIVERED` shall be `LEVEL2`

3.4.14 Function Spi_SetupEB

This function setup an external buffer to be used by a specific channel.

Prototype: `Std_ReturnType Spi_SetupEB(Spi_ChannelType Channel, const Spi_DataType *SrcDataBufferPtr, Spi_DataType *DesDataBufferPtr, Spi_NumberOfDataType Length);`

Table 3-21. Spi_SetupEB Arguments

Type	Name	Direction	Description
<code>Spi_ChannelType</code>	Channel	input	Channel ID
<code>const Spi_DataType *</code>	SrcDataBufferPtr	input	Pointer to the memory location that will hold the transmitted data
<code>Spi_NumberOfDataType</code>	Length	input	Length of the data in the external buffer
<code>Spi_DataType *</code>	DesDataBufferPtr	output	Pointer to the memory location that will hold the received data

Return: `Std_ReturnType`

Table 3-22. Spi_SetupEB Returns

Value	Description
<code>E_OK</code>	Setup command has been accepted
<code>E_NOT_OK</code>	Setup command has not been accepted

This function setup an external buffer to be used by a specific channel.

- Service ID: 0x05
- Sync/Async: Synchronous
- Reentrancy: Reentrant

pre:: The driver needs to be initialized before calling `Spi_SetupEB()` otherwise, the function `Spi_SetupEB()` shall raise the development error if `SPI_DEV_ERROR_DETECT` is `STD_ON`. Pre-compile parameter `SPI_CHANNEL_BUFFERS_ALLOWED` shall be `USAGE1` or `USAGE2`.

Note

Violates `Spi_c_REF_3` Violates MISRA 2004 Advisory Rule 11.4:
A cast should not be performed between a pointer to object type
and a different pointer to object type.

3.4.15 Function Spi_SyncTransmit

This function is used for synchronous transmission of a given sequence.

Prototype: `Std_ReturnType Spi_SyncTransmit(Spi_SequenceType Sequence);`

Table 3-23. Spi_SyncTransmit Arguments

Type	Name	Direction	Description
<code>Spi_SequenceType</code>	Sequence	input	Sequence ID

Return: `Std_ReturnType`

Table 3-24. Spi_SyncTransmit Returns

Value	Description
<code>E_OK</code>	Transmission command has been completed successfully
<code>E_NOT_OK</code>	Transmission command has not been accepted

This function is used for synchronous transmission of a given sequence.

- Service ID: 0x0a
- Sync/Async: Synchronous
- Reentrancy: Reentrant

pre:: The driver needs to be initialized before calling `Spi_SyncTransmit()`. otherwise, the function `Spi_SyncTransmit()` shall raise the development error if `SPI_DEV_ERROR_DETECT` is `STD_ON`. Pre-compile parameter `SPI_LEVEL_DELIVERED` shall be `LEVEL0` or `LEVEL2`

3.4.16 Function Spi_WriteIB

This function writes the given data into the buffer of a specific channel.

Prototype: `Std_ReturnType Spi_WriteIB(Spi_ChannelType Channel, const Spi_DataType *DataBufferPtr);`

Table 3-25. Spi_WriteIB Arguments

Type	Name	Direction	Description
Spi_ChannelType	Channel	input	Channel ID
const Spi_DataType *	DataBufferPtr	input	Pointer to source data buffer

Return: Std_ReturnType

Table 3-26. Spi_WriteIB Returns

Value	Description
E_OK	Command has been accepted
E_NOT_OK	Command has not been accepted

This function writes the given data into the buffer of a specific channel.

- Service ID: 0x02
- Sync/Async: Synchronous
- Reentrancy: Reentrant

pre.: The driver needs to be initialized before calling `Spi_WriteIB()` otherwise, the function `Spi_WriteIB()` shall raise the development error if `SPI_DEV_ERROR_DETECT` is `STD_ON`. Pre-compile parameter `SPI_CHANNEL_BUFFERS_ALLOWED` shall be `USAGE0` or `USAGE2`.

3.4.17 Function Spi_SetHWUnitAsyncMode

This function specifies the asynchronous mode for a given HWUnit.

Prototype: Std_ReturnType Spi_SetHWUnitAsyncMode(Spi_HWUnitType HWUnit, Spi_AsyncModeType AsyncMode);

Table 3-27. Spi_SetHWUnitAsyncMode Arguments

Type	Name	Direction	Description
Spi_HWUnitType	HWUnit	input	The ID of the HWUnit to be configured
Spi_AsyncModeType	AsyncMode	input	This parameter specifies the asynchronous operating mode (SPI_POLLING_MODE or SPI_INTERRUPT_MODE)

Return: Std_ReturnType

Table 3-28. Spi_SetHWUnitAsyncMode Returns

Value	Description
E_OK	The command ended successfully
E_NOT_OK	The command has failed

This function specifies the asynchronous mode for the SPI busses handled asynchronously. For synchronous HW units, the function has no impact. The function will fail in two cases:

- driver not initialised (SPI_E_UNINIT reported by DET)
- a sequence transmission is pending the the asynchronous HW unit (SPI_E_SEQ_PENDING reported by DET)

pre:: Pre-compile parameter SPI_LEVEL_DELIVERED shall be LEVEL2 and SPI_HWUNIT_ASYNC_MODE should be on STD_ON

3.4.18 Function Spi_SetClockMode

This function shall set different MCU clock configuration .

Prototype: Std_ReturnType Spi_SetClockMode(Spi_DualClockMode ClockMode);

Table 3-29. Spi_SetClockMode Arguments

Type	Name	Direction	Description
Spi_DualClockMode	ClockMode	input	Clock mode to be set (SPI_NORMAL SPI_ALTERNATE).

Return: Std_ReturnType

Table 3-30. Spi_SetClockMode Returns

Value	Description
E_OK	The driver is initialised and in an IDLE state. The clock mode can be changed.
E_NOT_OK	The driver is NOT initialised OR is NOT in an IDLE state. The clock mode can NOT be changed.

This function shall set different MCU clock configuration .

pre:: Pre-compile parameter SPI_DUAL_CLOCK_MODE shall be STD_ON.

3.4.19 Function Index

Table 3-31. Quick Function Reference

Type	Name	Arguments
Std_ReturnType	Spi_SetClockMode	Spi_DualClockMode ClockMode
Std_ReturnType	Spi_AsyncTransmit	Spi_SequenceType Sequence
void	Spi_Cancel	Spi_SequenceType Sequence
Std_ReturnType	Spi_Delnit	void
Spi_StatusType	Spi_GetAsyncStatus	void
Spi_StatusType	Spi_GetHWUnitStatus	Spi_HWUnitType HWUnit
Spi_JobResultType	Spi_GetJobResult	Spi_JobType Job
Spi_SeqResultType	Spi_GetSequenceResult	Spi_SequenceType Sequence
Spi_StatusType	Spi_GetStatus	void
void	Spi_GetVersionInfo	Std_VersionInfoType * VersionInfo
void	Spi_Init	const Spi_ConfigType * ConfigPtr
void	Spi_MainFunction_Handling	void
Std_ReturnType	Spi_ReadIB	Spi_ChannelType Channel Spi_DataType * DataBufferPtr
Std_ReturnType	Spi_SetAsyncMode	Spi_AsyncModeType AsyncMode
Std_ReturnType	Spi_SetHWUnitAsyncMode	Spi_HWUnitType HWUnit Spi_AsyncModeType AsyncMode
Std_ReturnType	Spi_SetupEB	Spi_ChannelType Channel const Spi_DataType * SrcDataBufferPtr Spi_NumberOfDataType Length Spi_DataType * DesDataBufferPtr
Std_ReturnType	Spi_SyncTransmit	Spi_SequenceType Sequence
Std_ReturnType	Spi_WriteIB	Spi_ChannelType Channel const Spi_DataType * DataBufferPtr

3.5 Enum Definitions

APIs of all enumerations supported by the driver are as per AUTOSAR SPI Driver software specification Version 4.0 Rev0003.

3.5.1 Enumeration Spi_DualClockMode

Specifies the Clock Modes.

Table 3-32. Enumeration Spi_DualClockMode Values

Value	Description
SPI_NORMAL = 0	Clock reference is from SpiClockRef.
SPI_ALTERNATE	Clock reference is from SpiAlternateClockRef.

3.5.2 Enumeration Spi_JobResultType

This type defines a range of specific Jobs status for SPI Driver.

Table 3-33. Enumeration Spi_JobResultType Values

Value	Description
SPI_JOB_OK = 0	The last transmission of the Job has been finished successfully.
SPI_JOB_PENDING	The SPI handler/Driver is performing a SPI Job.
SPI_JOB_FAILED	The last transmission of the Job has failed.
SPI_JOB_QUEUED	An asynchronous transmit Job has been accepted, while actual transmission for this Job has not started yet.

3.5.3 Enumeration Spi_SeqResultType

This type defines a range of specific Sequences status for SPI Driver.

Table 3-34. Enumeration Spi_SeqResultType Values

Value	Description
SPI_SEQ_OK = 0	The last transmission of the Sequence has been finished successfully.
SPI_SEQ_PENDING	The SPI handler/Driver is performing a SPI Sequence.
SPI_SEQ_FAILED	The last transmission of the Sequence has failed.
SPI_SEQ_CANCELLED	The last transmission of the Sequence has been cancelled by the user.

3.5.4 Enumeration Spi_StatusType

This type defines a range of specific status for SPI Driver.

Table 3-35. Enumeration Spi_StatusType Values

Value	Description
SPI_UNINIT = 0	Not initialized or not usable.
SPI_IDLE	Not currently transmitting any jobs.
SPI_BUSY	Is performing a SPI Job(transmit).

3.5.5 Enumeration Spi_AsyncModeType

This type defines a range of Async mode for SPI handler/Driver.

Table 3-36. Enumeration Spi_StatusType Values

Value	Description
SPI_POLLING_MODE = 0	The asynchronous mechanism is ensured by polling, so interrupts related to SPI busses handled asynchronously are disabled.
SPI_INTERRUPT_MODE	The asynchronous mechanism is ensured by interrupt, so interrupts related to SPI busses handled asynchronously are enabled.

3.5.6 Enumeration Spi_ModeType

This type Specifies the Clock Modes..

Table 3-37. Enumeration Spi_ModeType Values

Value	Description
SPI_MASTER = 0	SPI Hardware selected as MASTER.
SPI_SLAVE	SPI Hardware selected as SLAVE.

3.6 Structure Definitions

APIs of all structures supported by the driver are as per AUTOSAR SPI Driver software specification Version 4.0 Rev0003.

3.6.1 Structure Spi_HWUnitConfig

This structure holds the HWUnit configuration parameters.

Declaration

```
typedef struct
{
    uint8 Offset,
    uint32 IsSync,
    uint16 UseDma,
    uint8 TxDmaChannel,
    uint8 TxDmaChannelAux,
    uint8 RxDmaChannel
} Spi_HWUnitConfig;
```

Table 3-38. Structure Spi_HWUnitConfig member description

Member	Description
Offset	DSPI HWunit physical offset on SOC.
IsSync	Indicates if the HW unit is configured as Sync or Async.
UseDma	Boolean flag indicating if DMA will be used or not for this DSPI unit.
TxDmaChannel	Master TX DMA channel - enabled by the DSPI TX source.
TxDmaChannelAux	Auxiliary TX DMA channel - triggered by the master TX Dma.
RxDmaChannel	RX DMA channel - enabled by the DSPI RX source.

3.6.2 Structure Spi_ChannelConfig

The structure contains the channel configuration parameters.

Declaration

```
typedef struct
{
    Spi_BufferType BufferType,
    Spi_DataType DefaultTransmitValue,
    Spi_NumberOfDataType Length,
    const Spi_BufferDescriptor * BufferDescriptor,
    Spi_ChannelState * ChannelState
} Spi_ChannelConfig;
```

Table 3-39. Structure Spi_ChannelConfig member description

Member	Description
BufferType	Buffer Type IB/EB.
DefaultTransmitValue	Default Transmit Value.
Length	Data length.
BufferDescriptor	Buffer Descriptor.
ChannelState	Implementation specific field referencing the channel internal state.

3.6.3 Structure Spi_JobConfig

This is the structure containing all the parameters needed to completely define a Job.

Declaration

```
typedef struct
{
    Spi_ChannelType NumChannels,
    const Spi_ChannelType * ChannelIndexList,
    Spi_NotifyType * EndNotification,
    Spi_NotifyType * StartNotification,
    sint8 Priority,
    Spi_JobState * JobState,
    Spi_HWUnitType HWUnit,
    uint32 HWoffset,
    Spi_ExternalDeviceType ExternalDevice,
    Spi_LLD_DeviceAttributesConfig ExternalDeviceAttrs
} Spi_JobConfig;
```

Table 3-40. Structure Spi_JobConfig member description

Member	Description
NumChannels	Number of channels in the job.
ChannelIndexList	Channel index list.
EndNotification	Job end notification.
StartNotification	Job start notification.
Priority	Priority.
JobState	Implementation specific field referencing the channel internal state.
HWUnit	HWUnit.
HWoffset	HW Unit offset.
ExternalDevice	ExternalDevice.
ExternalDeviceAttrs	Implementation specific field: cached LLD device attributes.

3.6.4 Structure Spi_SequenceConfig

This structure contains all the needed data to configure one SPI Sequence.

Declaration

```
typedef struct
{
    Spi_JobType NumJobs,
    const Spi_JobType * JobIndexList,
    Spi_NotifyType * EndNotification,
    uint8 Interruptible
} Spi_SequenceConfig;
```

Table 3-41. Structure Spi_SequenceConfig member description

Member	Description
NumJobs	Number of jobs in the sequence.
JobIndexList	Job index list.
EndNotification	Job notification handler.
Interruptible	Boolean indicating if the Sequence is interruptible or not.

3.7 Define Definitions

APIs of all defines supported by the driver are as per AUTOSAR SPI Driver software specification Version 4.0 Rev0003.

3.7.1 Define SPI_ASYNCTRANSMIT_ID

API service ID for SPI async transmit function.

Definition: `#define SPI_ASYNCTRANSMIT_ID (uint8) 0x03u`

3.7.2 Define SPI_CANCEL_ID

API service ID for SPI cancel function.

Definition: `#define SPI_CANCEL_ID (uint8) 0x0Cu`

3.7.3 Define SPI_DEINIT_ID

API service ID for SPI DeInit function.

Definition: `#define SPI_DEINIT_ID (uint8) 0x01u`

3.7.4 Define SPI_GETHWUNITSTATUS_ID

API service ID for SPI get hwunit status function.

Definition: `#define SPI_GETHWUNITSTATUS_ID (uint8) 0x0Bu`

3.7.5 Define SPI_GETJOBRESULT_ID

API service ID for SPI get job result function.

Definition: `#define SPI_GETJOBRESULT_ID (uint8) 0x07u`

3.7.6 Define SPI_GETSEQUENCERESULT_ID

API service ID for SPI get sequence result function.

Definition: `#define SPI_GETSEQUENCERESULT_ID (uint8) 0x08u`

3.7.7 Define SPI_GETSTATUS_ID

API service ID for SPI get status function.

Definition: `#define SPI_GETSTATUS_ID (uint8) 0x06u`

3.7.8 Define SPI_GETVERSIONINFO_ID

API service ID for SPI get version info function.

Definition: `#define SPI_GETVERSIONINFO_ID (uint8) 0x09u`

3.7.9 Define SPI_INIT_ID

API service ID for SPI Init function.

Definition: `#define SPI_INIT_ID (uint8) 0x00u`

3.7.10 Define SPI_MAINFUNCTION_HANDLING_ID

API service ID for SPI main function.

Definition: `#define SPI_MAINFUNCTION_HANDLING_ID (uint8) 0x10u`

3.7.11 Define SPI_MODULE_ID

Definition: `#define SPI_MODULE_ID 83`

3.7.12 Define SPI_READIB_ID

API service ID for SPI read IB function.

Definition: `#define SPI_READIB_ID (uint8) 0x04u`

3.7.13 Define SPI_SETASYNCMODE_ID

API service ID for SPI set async mode function.

Definition: `#define SPI_SETASYNCMODE_ID (uint8) 0x0Du`

3.7.14 Define SPI_SETCLOCKMODE_ID

API service ID for SPI Set Clock Mode.

Definition: `#define SPI_SETCLOCKMODE_ID (uint8) 0x81u`

3.7.15 Define SPI_SETHWUNITASYNCMODE_ID

API service ID for SPI set HW Unit async mode.

Definition: `#define SPI_SETHWUNITASYNCMODE_ID (uint8) 0x80u`

3.7.16 Define SPI_SETUPEB_ID

API service ID for SPI setup EB function.

Definition: `#define SPI_SETUPEB_ID (uint8) 0x05u`

3.7.17 Define SPI_SYNCTRANSMIT_ID

API service ID for SPI sync transmit function.

Definition: `#define SPI_SYNCTRANSMIT_ID (uint8) 0x0Au`

3.7.18 Define SPI_VENDOR_ID

Definition: `#define SPI_VENDOR_ID 43`

3.7.19 Define SPI_WRITEIB_ID

API service ID for SPI write IB function.

Definition: `#define SPI_WRITEIB_ID (uint8) 0x02u`

3.7.20 Define SPI_E_ALREADY_INITIALIZED

API SPI_Init service called while the SPI driver has already been initialized.

Definition: `#define SPI_E_ALREADY_INITIALIZED (uint8) 0x4Au`

3.7.21 Define SPI_E_CONFIG_OUT_OF_RANGE

The number of sequences, jobs or channels exceeds precompile time sizes.

Definition: `#define SPI_E_CONFIG_OUT_OF_RANGE (uint8) 0x5Au`

3.7.22 Define SPI_E_JOB_EMPTY

No channel in job.

Definition: `#define SPI_E_JOB_EMPTY (uint8) 0x5Du`

3.7.23 Define SPI_E_PARAM_CHANNEL

API service called with wrong parameter.

Definition: `#define SPI_E_PARAM_CHANNEL (uint8) 0x0Au`

3.7.24 Define SPI_E_PARAM_EB_UNIT

When a sequence contains uninitialized external buffers.

Definition: `#define SPI_E_PARAM_EB_UNIT (uint8)0x5Bu`

3.7.25 Define SPI_E_PARAM_JOB

API service called with wrong parameter.

Definition: `#define SPI_E_PARAM_JOB (uint8)0x0Bu`

3.7.26 Define SPI_E_PARAM_LENGTH

API service called with wrong parameter.

Definition: `#define SPI_E_PARAM_LENGTH (uint8)0x0Du`

3.7.27 Define SPI_E_PARAM_SEQ

API service called with wrong parameter.

Definition: `#define SPI_E_PARAM_SEQ (uint8)0x0Cu`

3.7.28 Define SPI_E_PARAM_UNIT

API service called with wrong parameter.

Definition: `#define SPI_E_PARAM_UNIT (uint8)0x0Eu`

3.7.29 Define SPI_E_SEQ_EMPTY

No job in sequence.

Definition: `#define SPI_E_SEQ_EMPTY (uint8)0x5Cu`

3.7.30 Define SPI_E_SEQ_IN_PROCESS

Synchronous transmission service called at wrong time.

Definition: `#define SPI_E_SEQ_IN_PROCESS (uint8)0x3Au`

3.7.31 Define SPI_E_SEQ_PENDING

Services called in a wrong sequence.

Definition: `#define SPI_E_SEQ_PENDING (uint8)0x2Au`

3.7.32 Define SPI_E_UNINIT

API service used without module initialization.

Definition: `#define SPI_E_UNINIT (uint8)0x1Au`

3.8 Symbolic Names DISCLAIMER

All containers having the symbolic name tag set as true in the Autosar schema will generate defines like:

```
#define <Container_Short_Name> <Container_ID>
```

For this reason it is forbidden to duplicate the name of such containers across the MCAL configuration, or to use names that may trigger other compile issues (e.g. match existing `#ifdefs` arguments).

3.9 Configuration Parameters

As per the AUTOSAR specification the driver has two types of configurations parameters – Pre-Compile parameters and Post-Build parameters. Pre-Compile parameters are stored in the file “Spi_Cfg.h” & “Spi_Cfg.c”. The Post-Build parameters are stored in the file “Spi_PBcfg.c”.

The files to be used for different configuration types are listed below.

- 1) Variant PC: Spi_Cfg.h, Spi_Cfg.c
- 2) Variant PB: Spi_Cfg.h, Spi_PBcfg.c
- 3) Variant LT: Spi_Cfg.h, Spi_Lcfg.c

A section for Spi_PBCfg.c file is needed in linker file to place the post build configuration in desired location. Please refer to section-8 of "AUTOSAR_SWS_C_ImplementationRules.pdf" for complete details on configuration types.

3.9.1 Pre-Compile Parameters

Pre-Compile parameters, their possible values and meaning are described in the following text. Pre-Compile parameters are implemented as preprocessor defines.

The configuration screens for spi precompile parameters in Tresos® Studio configuration tool from Tresos Elektrobit Tresos Release 14.2.1 (b140128-1223) is given below:

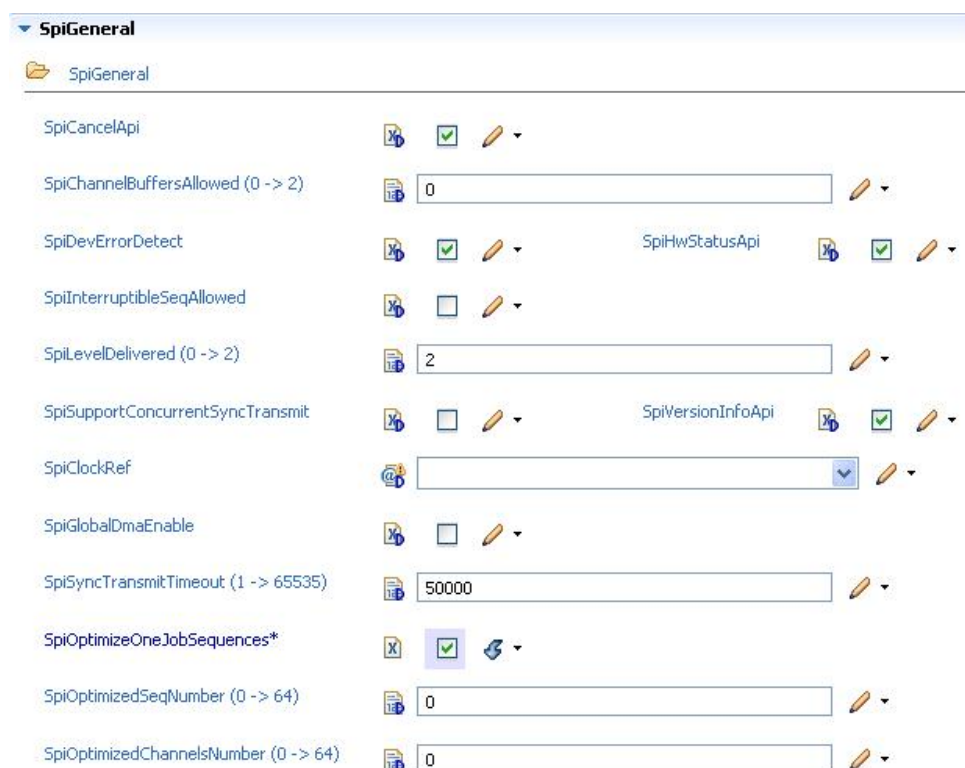


Figure 3-1. SpiGeneral Configuration

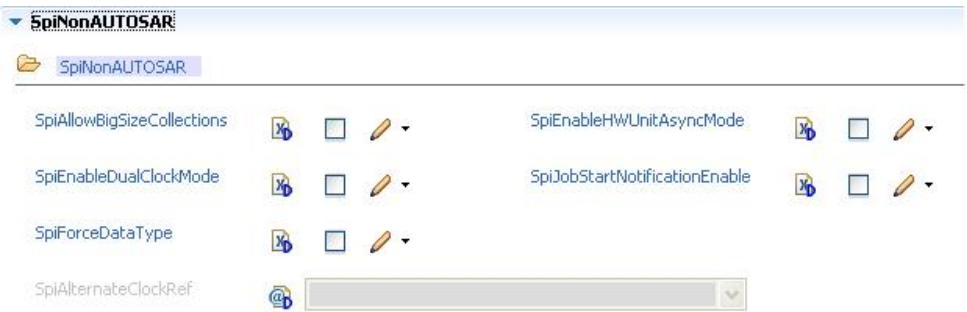
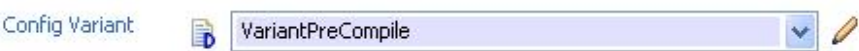


Figure 3-2. SpiNonAutosar Configuration

Note

If pre-compile variant is preferred,

- IMPLEMENTATION_CONFIG_VARIANT in Tresos GUI should be selected as “VariantPreCompile” as shown below.
- 
- Spi_Cfg.c should be compiled
- Spi_PBcfg.c should not be compiled

3.9.1.1 SpiCancelApi

Table 3-42. SpiCancelApi

Description	Defines whether the function Spi_Cancel shall be available (STD_ON) or not (STD_OFF) at compile time.
Class	Autosar Parameter
Range	True, false

Table continues on the next page...

Table 3-42. SpiCancelApi (continued)

Default	true
Source File	Spi_Cfg.h
Source Representation	#define SPI_CANCEL_API <STD_OFF, STD_ON>

3.9.1.2 SpiChannelBuffersAllowed

Table 3-43. SpiChannelBuffersAllowed

Description	USAGE0: Spi handler/driver manages only Internal Buffers USAGE1: Spi handler/driver manages only External Buffers USAGE2: Spi handler/driver manages both Buffers types.
Class	Autosar Parameter
Range	USAGE0, USAGE1 or USAGE2
Default	USAGE0
Source File	Spi_Cfg.h
Source Representation	#define SPI_CHANNEL_BUFFERS_ALLOWED<USAGE0, USAGE1, USAGE2>

3.9.1.3 SpiDevErrorDetect

Table 3-44. SpiDevErrorDetect

Description	Defines whether support of development error detection should be included at compile time (STD_ON) or excluded (STD_OFF). When enabled API parameter checking is done at runtime.
Class	Autosar Parameter
Range	True,false
Default	false
Source File	Spi_Cfg.h
Source Representation	#define Spi_DEV_ERROR_DETECT<STD_OFF, STD_ON>

3.9.1.4 SpiHWStatusAPI

Table 3-45. SpiHWStatusAPI

Description	Defines whether function “Spi_GetHWUnitStatus” is available (STD_ON) or unavailable (STD_OFF) at compile time.
Class	Autosar Parameter
Range	True, false

Table continues on the next page...

**Table 3-45. SpiHWStatusAPI
(continued)**

Default	true
Source File	Spi_Cfg.h
Source Representation	#define SPI_HW_STATUS_API<STD_OFF, STD_ON>

3.9.1.5 SpiInterruptibleSeqAllowed

Table 3-46. SpiInterruptibleSeqAllowed

Description	Defines whether a sequence should be interruptible at compile time (STD_ON) or excluded (STD_OFF).
Class	Autosar Parameter
Range	True, false
Default	true
Source File	Spi_Cfg.h
Source Representation	#define SPI_INTERRUPTIBLE_SEQ_ALLOWED<STD_OFF, STD_ON>

3.9.1.6 SpiLevelDelivered

Table 3-47. SpiLevelDelivered

Description	Defines the type of communication that the Spi handler/driver will use. LEVEL0: Only Simple synchronous communication is allowed. LEVEL1: Basic Asynchronous communication is allowed. A priority policy is applied using this configuration level. LEVEL2: Using this level the Spi communication will be based on asynchronous behavior or synchronous handling. Using this level an interrupt or polling mechanism can be used.
Class	Autosar Parameter
Range	LEVEL0, LEVEL1 & LEVEL2
Default	LEVEL2
Source File	Spi_Cfg.h
Source Representation	#define SPI_LEVEL_DELIVERED< LEVEL0, LEVEL1, LEVEL2>

Note

SPI_LEVEL_0 is sufficient when Synchronous transmission is used. Spi_Irq.c is not needed if only synchronous transmission is used. The DMA functionality is not used when level 0 is selected. The DMA can be used only in level 1 or 2 and when the SpiPhyUnitAsyncMethod is set to DMA.

3.9.1.7 SpiVersionInfoApi

Table 3-48. SpiVersionInfoApi

Description	Defines whether driver function will be included at compile time (STD_ON) or excluded (STD_OFF).
Class	Autosar Parameter
Range	True, false
Default	true
Source File	Spi_Cfg.h
Source Representation	#define SPI_VERSION_INFO_API<STD_OFF, STD_ON>

3.9.1.8 SpiClockRef

Table 3-49. SpiClockRef

Description	The reference to MCU ClockReference Point to get the SPI clock. In EB tresos configuration, the DSPIs should use a clock reference point with McuPeripheralClkSet_1 (Peripheral set 2) source for SpiClockRef. The reference points in Mcu module match with Reference Manual as follow: McuPeripheralClkSet_0 - Peripheral set 1 McuPeripheralClkSet_1 - Peripheral set 2 McuPeripheralClkSet_2 - Peripheral set 3
Class	Implementation specific parameter
Range	N/A
Default	N/A
Source File	Spi_Cfg.h
Source Representation	It is only used in code template.

3.9.1.9 SpiGlobalDmaEnable

Table 3-50. SpiGlobalDmaEnable

Description	This parameter is used only for Asynchronous Transmission. If enabled, the DSPI units working in asynchronous transferring mode may be configured in DMA operation mode or in PIO_FIFO mode according to the value of the SpiPhyUnitAsyncMethod field. If disabled, all transfers will be made in PIO_FIFO mode. DMA - DMA is used as transfer method for transferring data from channel buffer to DSPI Transmit FIFO register and receives data from DSPI Receive FIFO register to channel buffer.
--------------------	---

Table continues on the next page...

Table 3-50. SpiGlobalDmaEnable (continued)

	PIO_FIFO - Programmed Input Output First in First Out – The Spi driver transfers a maximum of four data entries from channel buffer to DSPI Transmit FIFO registers and receives a maximum of four data entries from DSPI Receive FIFO registers to channel buffer.
Class	Implementation specific parameter
Range	true, false
Default	true
Source File	Spi_Cfg.h
Source Representation	#define SPI_DMA_USED <STD_OFF, STD_ON>

3.9.1.10 SpiSyncTransmitTimeout

Table 3-51. SpiSyncTransmitTimeout

Description	Timeout value (in microseconds) used by SPI_SyncTransmit function to wait for TX/RX FIFO to be emptied/filled. When this timeout is elapsed, an error is reported to the Dem module
Class	Implementation specific parameter
Range	1-65535
Default	50000
Source File	Spi_Cfg.h
Source Representation	#define SPI_TIMEOUT_COUNTER <number_of_waiting_loops>

Note

The precision of this value is quite low, as the time expressed in microseconds is converted in number of waiting loops and the duration of a loop cycle depends of compiler configuration. You must configure this timeout at a value much larger than the maximum transmission latency used in the configuration. Ex: Supposing that the slowest transmission is for a device accepting frames of 16 bits at a baud rate of 500KHz, and demanding one Clk before the CS activation and after CS releasing - Frame transmissions duration will be: $2\mu s T_{asc} + 2\mu s \times 16 + 2\mu s T_{pcssck} = 36\mu s$ - You may configure SpiSyncTransmitTimeout at 360 μs (10 times the maximum latency duration)

3.9.1.11 SpiOptimizeOneJobSequences

Table 3-52. SpiOptimizeOneJobSequences

Description	Activates the SPI transmission optimization for the sequences having only one job. At his moment, the optimization is in place only for synchronous transmissions.
Class	Implementation specific parameter
Range	true, false
Default	false
Source File	Spi_Cfg.h
Source Representation	#define SPI_OPTIMIZE_ONE_JOB_SEQUENCES (<STD_OFF, STD_ON>)

Note

This option requires additional RAM for two internal buffers (for sequences having only one job, and for their linked channels), in order to cache configuration information to be used during transmission initialization. During Spi_Init() a caching action is performed on all sequences having only one job

3.9.1.12 SpiOptimizedChannelsNumber

Table 3-53. SpiOptimizedChannelsNumber

Description	The maximum number of channels in expected sequences with one job, targeted for optimization. If, for a given configuration, the number of channels in sequences having only one job exceeds this value, the sequences will be optimized only in the limit of 'SpiOptimizedChannelsNumber' consumption.
Class	Implementation specific parameter
Range	0-64
Default	0 (the total number of channels in one job sequences in configuration)
Source File	Spi_Cfg.h
Source Representation	#define SPI_OPTIMIZED_CHANNEL_BUFFER_SIZE (<Channel_Count>)

Note

A value of 0 will define the buffer size for fitting all optimize-able sequences

3.9.1.13 SpiOptimizedSeqNumber

Table 3-54. SpiOptimizedSeqNumber

Description	The maximum number of expected sequences with one job, targeted for optimization. If, for a given configuration, the number of sequences having only one job exceeds this value, only first 'SpiOptimizedSeqNumber' sequences will operate in optimized transmission mode.
Class	Implementation specific parameter
Range	0-64
Default	0 (the total number of one job sequences in configuration)
Source File	Spi_Cfg.h
Source Representation	#define SPI_OPTIMIZED_SEQ_BUFFER_SIZE (<Sequence_Count>)

Note

A value of 0 will define the buffer size for fitting all optimize-able sequences

3.9.1.14 SpiEnableHWUnitAsyncMode

Table 3-55. SpiEnableHWUnitAsyncMode

Description	Enable Spi_SetHWUnitAsyncMode() function, which allows defining distinct operation mode (POLLING or INTERRUPT) for each HWUnit.
Class	Non AUTOSAR Requirement
Range	true, false
Default	false
Source File	Spi_Cfg.h
Source Representation	#define SPI_HWUNIT_ASYNC_MODE (<STD_OFF, STD_ON>)

Note

This feature is not required by Autosar, which defines asynchronous mode configuration at driver level only

3.9.1.15 SpiEnableDualClockMode

Table 3-56. SpiEnableDualClockMode

Description	Enable Spi_SetClockMode() function, which allows dual MCU clock configuration settings.
Class	Implementation specific parameter
Range	true, false
Default	false

Table continues on the next page...

Table 3-56. SpiEnableDualClockMode (continued)

Source File	Spi_Cfg.h
Source Representation	#define SPI_DUAL_CLOCK_MODE (<STD_OFF, STD_ON>)

3.9.1.16 SpiAlternateClockRef

Table 3-57. SpiAlternateClockRef

Description	Reference to the alternate clock configuration, retrieved from the MCU plugin.
Class	Implementation specific parameter
Range	N/A
Default	N/A
Source File	Spi_Cfg.h
Source Representation	It is only used in code template.

3.9.1.17 SpiAllowBigSizeCollections

Table 3-58. SpiAllowBigSizeCollections

Description	Reference to the Big Size Collections configuration
Class	Implementation specific parameter
Range	N/A
Default	N/A
Source File	Spi_Cfg.h
Source Representation	#define SPI_ALLOW_BIGSIZE_COLLECTIONS (STD_OFF)

3.9.1.18 SpiForceDataType

Table 3-59. SpiForceDataType

Description	This parameter defines the type of Spi_DataType as uint16 (STD_OFF) or uint8 (STD_ON). In both cases, it allows mixing the channels having different size (UINT16/UINT8) .
Class	Implementation specific parameter
Range	True, false
Default	False
Source File	Spi_Cfg.h

Table continues on the next page...

Table 3-59. SpiForceDataType (continued)

Source Representation	#define SPI_FORCE_DATA_TYPE (STD_OFF)
-----------------------	---------------------------------------

Note:

If **SpiForceDataType** is true (Spi_DataType defined as UINT8), in case of external buffers (EB) corresponding to channels that have SpiDataWidth between 9 and 16 bits, the EB buffers must be aligned in memory at even locations.

3.9.2 Post-Build parameters

Post-Build parameters, their possible values and their meaning are described in the following text. The Post-Build parameters are implemented as constant structures and arrays stored in flash memory of the MCU.

The configuration screen for SPI postbuild parameters in Tresos® Studio configuration tool from Tresos Elektrobit Tresos Release 14.2.1 (b140128-1223) is given below:

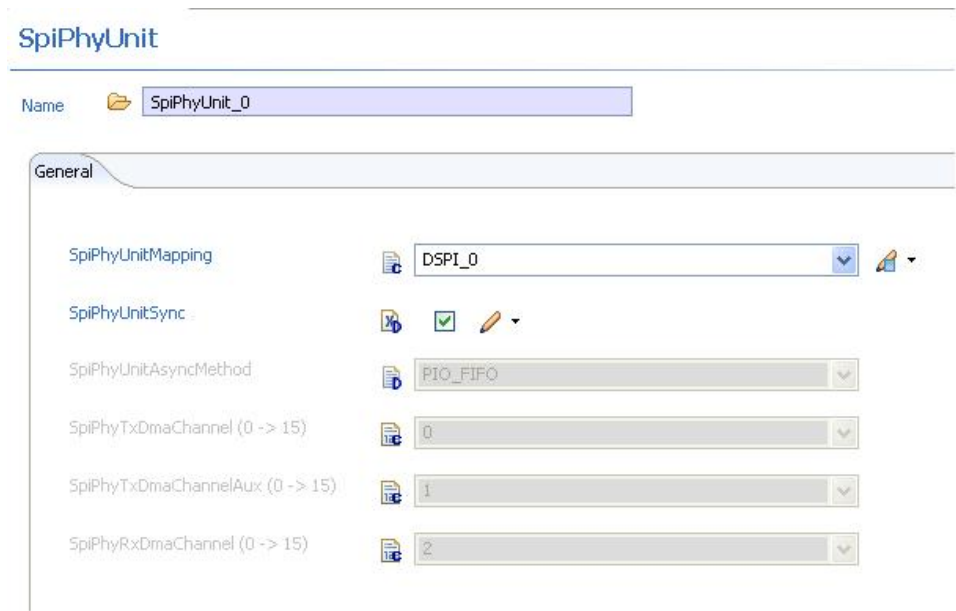
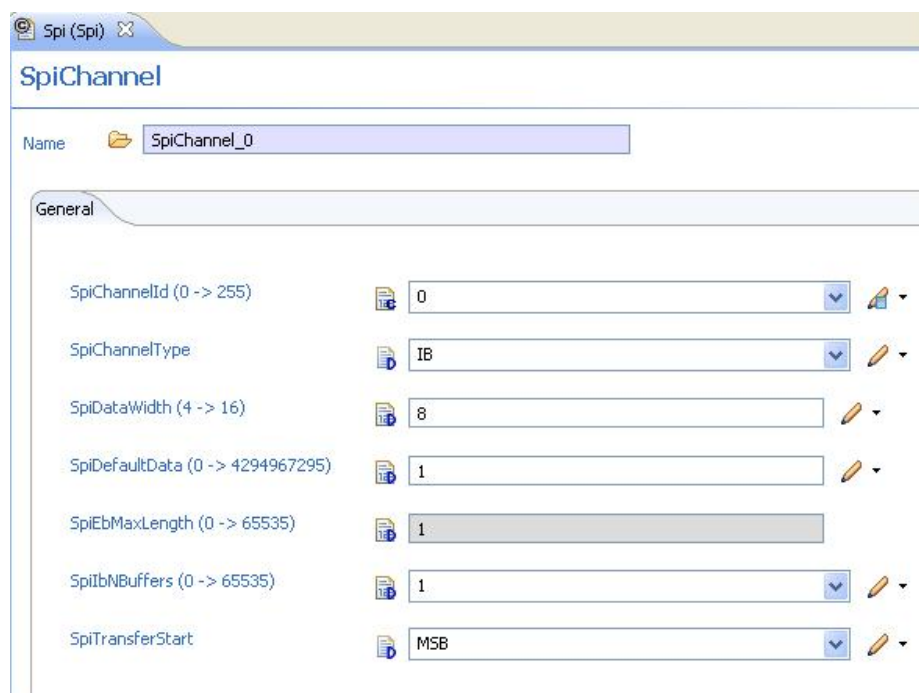


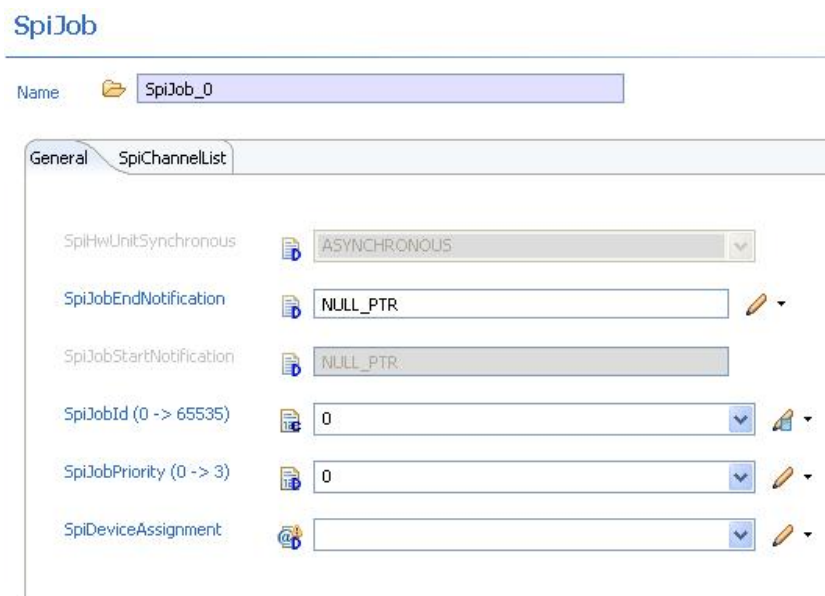
Figure 3-3. SpiPhyUnit Configuration



The SpiChannel configuration window shows the following settings:

Property	Value
Name	SpiChannel_0
SpiChannelId (0 -> 255)	0
SpiChannelType	IB
SpiDataWidth (4 -> 16)	8
SpiDefaultData (0 -> 4294967295)	1
SpiEbMaxLength (0 -> 65535)	1
SpiNbBuffers (0 -> 65535)	1
SpiTransferStart	MSB

Figure 3-4. SpiChannel Configuration



The SpiJob configuration window shows the following settings:

Property	Value
Name	SpiJob_0
SpiHwUnitSynchronous	ASYNCHRONOUS
SpiJobEndNotification	NULL_PTR
SpiJobStartNotification	NULL_PTR
SpiJobId (0 -> 65535)	0
SpiJobPriority (0 -> 3)	0
SpiDeviceAssignment	

Figure 3-5. SpiJob Configuration

SpiSequence

Name  SpiSequence_0

General SpiJobAssignment

SpiInterruptibleSequence  ☐ 

SpiSeqEndNotification  NULL_PTR 

SpiSequenceId (0 -> 255)  0  

Figure 3-6. SpiSequence Configuration

SpiExternalDevice

Name: SpiExternalDevice_0

General

- SpiBaudrate (4 -> 16000000): 100000.0
- SpiCsIdentifier: PCS0
- SpiCsSelection: CS_VIA_PERIPHERAL_ENGINE
- SpiCsPolarity: HIGH
- SpiDataShiftEdge: LEADING
- SpiEnableCs: ☒
- SpiHwUnit: CSIB0
- SpiShiftClockIdleLevel: HIGH
- SpiTimeClk2Cs (0 -> 0.0001): 1.0E-6
- SpiTimeCs2Clk (0 -> 0.01): 1.0E-6
- SpiTimeCs2Cs (0 -> 0.01): 6.4E-6
- SpiCsContinuous: true

Figure 3-7. SpiExternalDevice Configuration

3.9.2.1 SpiPhyUnitMapping

Table 3-60. SpiPhyUnitMapping

Description	Logical SpiHwUnit to physical DSPI_[0 1 2 3 4 5] assignment.
Class	Autosar parameter
Range	DSPI_0, DSPI_1, DSPI_2, DSPI_3, DSPI_4, DSPI_5
Default	DSPI_0
Source File	-
Source Representation	The bold section of the following structure <pre>static VAR(Spi_LLD_DeviceType, SPI_VAR) DspiDevice_PB[SPI_MAX_HWUNIT] = { DSPI_0_OFFSET, SPI_PHYUNIT_ASYNC, (uint16) TRUE, 3u, 4u,</pre>

Table 3-60. SpiPhyUnitMapping

	5u },
--	----------

3.9.2.2 SpiPhyUnitSync

Table 3-61. SpiPhyUnitSync

Description	This parameter is used to specify if this HwUnit can do only synchronous transfers. If true then this hardware unit is dedicated for Synchronous transfers. If false then this hardware unit is dedicated for Asynchronous transfers.
Class	Autosar parameter
Range	True, false
Default	True
Source File	Spi_PbCfg.c
Source Representation	The bold section of the following structure <pre>static VAR(Spi_LLD_DeviceType, SPI_VAR) DspiDevice_PB[SPI_MAX_HWUNIT] = { { DSPI_0_OFFSET, SPI_PHYUNIT_ASYNC, (uint16) TRUE, 3u, 4u, 5u }, ... },</pre>

Note

Asynchronous Transfer (false) is applicable only if SpiLevelDelivered is configured for LEVEL1 or LEVEL2 and Synchronous Transfer (true) is applicable only if SpiLevelDelivered is configured for LEVEL0.

3.9.2.3 SpiPhyUnitMode

Table 3-62. SpiPhyUnitMode

Description	Logical SpiHWunit to physical DSPI_[0 1 2 3 4 5] assignment. It depends on the number of units present in the chip version. This is an implementation specific parameter.
Class	Autosar parameter
Range	SPI_MASTER, SPI_SLAVE
Default	SPI_MASTER
Source File	Spi_Cfg.c
Source Representation	

3.9.2.4 SpiPhyUnitAsyncMethod

Table 3-63. SpiPhyUnitAsyncMethod

Description	This parameter is used only for Asynchronous Transmission. Select one of Asynchronous Transfer Method for Spi: DMA and PIO with FIFO. DMA - DMA is used as transfer method for transferring data from channel buffer to DSPI Transmit FIFO register and receives data from DSPI Receive FIFO register to channel buffer. PIO_FIFO - Programmed Input Output First in First Out – The Spi driver transfers a maximum of four data entries from channel buffer to DSPI Transmit FIFO registers and receives a maximum of four data entries from DSPI Receive FIFO registers to channel buffer.
Class	Implementation specific parameter
Range	DMA, PIO_FIFO
Default	PIO_FIFO
Source File	Spi_Cfg.h
Source Representation	The bold section of the following structure <pre>static VAR(Spi_LLD_DeviceType, SPI_VAR) DspiDevice_PB[SPI_MAX_HWUNIT] = { { DSPI_0_OFFSET, SPI_PHYUNIT_ASYNC, (uint16)TRUE, 3u, 4u, 5u }, ... }</pre>

Note

For Asynchronous Transmission Notification function, SPI_JOB_FAILED will not be called from Spi Hardware since there are no conditions to detect a hardware failure and make the Job status as failed.

3.9.2.5 SpiPhyTxDmaChannel

Table 3-64. SpiPhyTxDmaChannel

Description	DSPI Master Transmit DMA Channel, used to prepare the DSPI transmission dataframes starting from the TX buffer content. A maximum of 0 to 15 channels could be configured.
Class	Implementation specific parameter
Range	-
Default	(DSPI_index x 3) + 0

Table continues on the next page...

Table 3-64. SpiPhyTxDmaChannel (continued)

Source File	Spi_Cfg.c, Spi_PBcfg.c, Spi_LTcfg.c
Source Representation	The bold section of the following structure <pre>static VAR(Spi_LLD_DeviceType, SPI_VAR) DspiDevice_PB[SPI_MAX_HWUNIT] = { { DSPI_0_OFFSET, SPI_PHYUNIT_ASYNC, (uint16)TRUE, 3u, 4u, 5u }, </pre>

Note

SpiPhyTxDmaChannel is applicable only if DMA is selected for Asynchronous Transfer Method (SpiPhyUnitAsyncMethod).

3.9.2.6 SpiPhyTxDmaChannelAux**Table 3-65. SpiPhyTxDmaChannelAux**

Description	DSPI Auxiliary Transmit DMA Channel, used to place dataframes into the DSPI registers. A maximum of 0 to 15 channels could be configured.
Class	Implementation specific parameter
Range	-
Default	(DSPI_index x 3) + 1
Source File	Spi_Cfg.c, Spi_PBcfg.c, Spi_LTcfg.c
Source Representation	The bold section of the following structure <pre>static VAR(Spi_LLD_DeviceType, SPI_VAR) DspiDevice_PB[SPI_MAX_HWUNIT] = { { DSPI_0_OFFSET, SPI_PHYUNIT_ASYNC, (uint16)TRUE, 3u, 4u, 5u }, </pre>

Note

SpiPhyTxDmaChannelAux is applicable only if DMA is selected for Asynchronous Transfer Method (SpiPhyUnitAsyncMethod).

3.9.2.7 SpiPhyRxDmaChannel

Table 3-66. SpiPhyRxDmaChannel

Description	DSPI Receive DMA Channel, used to read the deserialized dataframes into the RX buffers. A maximum of 0 to 15 channels could be configured.
Class	Implementation specific parameter
Range	-
Default	(DSPI_index x 3) + 2
Source File	Spi_Cfg.c, Spi_PBcfg.c, Spi_LTcfg.c
Source Representation	The bold section of the following structure <pre>static VAR(Spi_LLD_DeviceType, SPI_VAR) DspiDevice_PB[SPI_MAX_HWUNIT] = { { DSPI_0_OFFSET, SPI_PHYUNIT_ASYNC, (uint16)TRUE, 3u, 4u, 5u }, </pre>

Note

SpiPhyRxDmaChannel is applicable only if DMA is selected for Asynchronous Transfer Method (SpiPhyUnitAsyncMethod).

3.9.2.8 SpiMaxChannel

Table 3-67. SpiMaxChannel

Description	This parameter contains the number of Channels configured. It will be gathered by tools during the configuration stage.
Class	Autosar Parameter
Range	-
Default	-
Source File	-
Source Representation	<pre>/* SpiDriverConfig_PB Configuration */ CONST(Spi_ConfigType, SPI_CONST) SpiDriverConfig_PB = { 1, 6, 5, 4, SpiChannelConfig_PB0, SpiJobConfig_PB0, SpiSequenceConfig_PB0, &SpiAttributesConfig_PB0, HWUnitConfig_PB };</pre>

Note

SpiMaxChannel parameter is not used, instead max channel value is derived from number of channels configured.

SpiMaxChannel parameter is retained in configuration for consistency with AUTOSAR_EcucParamDef.xml rev 003.

3.9.2.9 SpiChannelId**Table 3-68. SpiChannelId**

Description	Channel unique identifier in the configuration.
Class	Autosar Parameter
Range	0 – (SpiMaxChannel-1)
Default	0
Source File	-
Source Representation	#define <ChannelName> (Spi_ChannelType)<SpiChannelId>

3.9.2.10 SpiChannelType**Table 3-69. SpiChannelType**

Description	Buffer usage with EB/IB channel
Class	Autosar Parameter
Range	IB , EB
Default	IB
Source File	-
Source Representation	The bold section of following structure: <pre> /* Channel Configuration */ static CONST(Spi_ChannelConfig, SPI_CONST) SpiChannelConfig_PB0[6] = { { /* CH_WREN_CMD_EEPROM1*/ IB, 6 1, &Buffer_CH_WREN_CMD_EEPROM1, &SpiChannelState[0] }, }; </pre>

3.9.2.11 SpiDataWidth

Table 3-70. SpiDataWidth

Description	This parameter is the width of a data unit to be transmitted.
Class	Autosar Parameter
Range	4...16
Default	8
Source File	Spi_PbCfg.c
Source Representation	The bold section of the following structure: <pre> static CONST(Spi_LLD_ChannelAttributesConfig,SPI_CONST) DspiChannelAttributesConfig_PB0[6] = { DSPI_CMD_MSB DSPI_CMD_WIDTH_8, DSPI_CMD_MSB DSPI_CMD_WIDTH_8, DSPI_CMD_MSB DSPI_CMD_WIDTH_8, DSPI_CMD_MSB DSPI_CMD_WIDTH_8 } </pre>

3.9.2.12 SpiDefaultData

Table 3-71. SpiDefaultData

Description	This parameter is the default value that is transmitted.
Class	Autosar Parameter
Range	-
Default	1
Source File	Spi_PbCfg.c
Source Representation	The bold section of following structure: <pre> /* Channel Configuration */ static CONST(Spi_ChannelConfig, SPI_CONST) SpiChannelConfig_PB0[6] = { { /* CH_WREN_CMD_EEPROM1*/ IB, 6, 1, &Buffer_CH_WREN_CMD_EEPROM1, &SpiChannelState[0] } } </pre>

3.9.2.13 SpiEbMaxlength

Table 3-72. SpiEbMaxlength

Description	This parameter contains the maximum size of data buffers in case of EB Channels only.
Class	Autosar Parameter
Range	-
Default	1
Source File	Spi_PbCfg.c
Source Representation	The bold section of following structure: <pre> /* Channel Configuration */ static CONST(Spi_ChannelConfig, SPI_CONST) SpiChannelConfig_PB0[6] = { { /* CH_WREN_CMD_EEPROM1*/ EB, 6, 1, &Buffer_CH_WREN_CMD_EEPROM1, &SpiChannelState[0] }, }; </pre>

3.9.2.14 SpilbNBuffers

Table 3-73. SpilbNBuffers

Description	This parameter contains the maximum number of data buffers in case of IB Channels and only.
Class	Autosar Parameter
Range	1 to 65535
Default	65535
Source File	Spi_PbCfg.c
Source Representation	The bold section of following structure: <pre> /* Channel Configuration */ static CONST(Spi_ChannelConfig, SPI_CONST) SpiChannelConfig_PB0[6] = { { /* CH_WREN_CMD_EEPROM1*/ IB, 6, 1, &Buffer_CH_WREN_CMD_EEPROM1, &SpiChannelState[0] }, }; </pre>

3.9.2.15 SpiTransferStart

Table 3-74. SpiTransferStart

Description	This parameter defines the first starting bit for transmission.
Class	Autosar Parameter
Range	-
Default	-
Source File	Spi_PbCfg.c
Source Representation	The bold section of the following structure: <pre>static CONST(Spi_LLD_ChannelAttributesConfig,SPI_CONST) DspiChannelAttributesConfig_PB0[6] = { DSPI_CMD_MSB DSPI_CMD_WIDTH_8, DSPI_CMD_MSB DSPI_CMD_WIDTH_8, DSPI_CMD_MSB DSPI_CMD_WIDTH_8, DSPI_CMD_MSB DSPI_CMD_WIDTH_8, }</pre>

3.9.2.16 SpiBaudRate

Table 3-75. SpiBaudRate

Description	This parameter is the communication baudrate – This parameter allows using a range of values, from the point of view of configuration tools. SpiBaudrate is dependant on Processor Frequency. The minimum baud rate at 4Mhz Processor frequency is 18 bits / second (18 bits/s) and the maximum baud rate at 64 MHz Processor frequency is 10,700,000 bits/second (10.7Mb/s)
Class	Autosar Parameter
Range	18 bits/second – 16,000,000 bits/second
Default	100000
Source File	Spi_PbCfg.c
Source Representation	The bold section of the following structure <pre>static CONST(Spi_LLD_DeviceAttributesConfig,SPI_CONST) DspiDeviceAttributesConfig_PB0[1] = { { (uint32)(DSPI_CTAR_CPOL HIGH DSPI_CTAR_CPHA_LEADING DSPI_CTAR_PCCLK_3 DSPI_CTAR_SCCLK_64 DSPI_CTAR_PASC_3 DSPI_CTAR_SASC_128 DSPI_CTAR_PDT_3 DSPI_CTAR_SDT_64 DSPI_CTAR_PBR_5 DSPI_CTAR_SBR_16 DSPI_CTAR_DBR_0), (uint32)(DSPI_CMD_PCS0 /* Chip Select Pin */ DSPI_CMD_CONTINUOUS_TRUE), /* Continous chip select*/ (uint32)DSPI_PCS0_IDLEHIGH /* Chip select polarity */ }, };</pre>

3.9.2.17 SpiCs

Table 3-76. SpiCs

Description	This parameter defines the Chip select pin
Class	Autosar Parameter
Range	PCS0 to PCS1
Default	PCS0
Source File	Spi_PbCfg.c
Source Representation	The bold section of the following structure <pre>static CONST(Spi_LLD_DeviceAttributesConfig,SPI_CONST) DspiDeviceAttributesConfig_PB0[1] = { { (uint32)(DSPI_CTAR_CPOL_HIGH DSPI_CTAR_CPHA_LEADING DSPI_CTAR_PCCLK_3 DSPI_CTAR_SCCLK_64 DSPI_CTAR_PASC_3 DSPI_CTAR_SASC_128 DSPI_CTAR_PDT_3 DSPI_CTAR_SDT_64 DSPI_CTAR_PBR_5 DSPI_CTAR_SBR_16 DSPI_CTAR_DBR_0), (uint32)(DSPI_CMD_PCS0 /* Chip Select Pin */ DSPI_CMD_CONTINOUS_TRUE),/* Continous chip select */ (uint32)DSPI_PCS0_IDLEHIGH/* Chip select polarity */ }, };</pre>

3.9.2.18 SpiCsPolarity

Table 3-77. SpiCsPolarity

Description	This parameter defines the active polarity of Chip Select.
Class	Autosar Parameter
Range	High / Low
Default	High
Source File	Spi_PbCfg.c
Source Representation	The bold section of the following structure: <pre>static CONST(Spi_LLD_DeviceAttributesConfig,SPI_CONST) DspiDeviceAttributesConfig_PB0[1] = { { (uint32)(DSPI_CTAR_CPOL_HIGH DSPI_CTAR_CPHA_LEADING DSPI_CTAR_PCCLK_3 DSPI_CTAR_SCCLK_64 DSPI_CTAR_PASC_3 DSPI_CTAR_SASC_128 DSPI_CTAR_PDT_3 DSPI_CTAR_SDT_64 DSPI_CTAR_PBR_5 DSPI_CTAR_SBR_16 DSPI_CTAR_DBR_0), (uint32)(DSPI_CMD_PCS0 /* Chip Select Pin */ DSPI_CMD_CONTINOUS_TRUE), /* Continous chip select */ </pre>

Table 3-77. SpiCsPolarity

	<pre> (uint32)DSPI_PCS0_IDLEHIGH /* Chip select polarity */ }, }; </pre>
--	--

3.9.2.19 SpiDataShiftEdge

Table 3-78. SpiDataShiftEdge

Description	This parameter defines the SPI data shift edge.
Class	Autosar Parameter
Range	Leading / Trailing
Default	Leading
Source File	Spi_PbCfg.c
Source Representation	The bold section of the following structure <pre> static CONST(Spi_LLD_DeviceAttributesConfig, SPI_CONST) DspiDeviceAttributesConfig_PB0[1] = { { (uint32)(DSPI_CTAR_CPOL_HIGH DSPI_CTAR_CPHA_LEADING DSPI_CTAR_PCSCLK_3 DSPI_CTAR_SCSCCLK_64 DSPI_CTAR_PASC_3 DSPI_CTAR_SASC_128 DSPI_CTAR_PDT_3 DSPI_CTAR_SDT_64 DSPI_CTAR_PBR_5 DSPI_CTAR_SBR_16 DSPI_CTAR_DBR_0), (uint32)(DSPI_CMD_PCS0 /* Chip Select Pin */ DSPI_CMD_CONTINUOUS_TRUE), /* Continous chip select */ (uint32)DSPI_PCS0_IDLEHIGH /* Chip select polarity */ } }; </pre>

3.9.2.20 SpiEnableCs

Table 3-79. SpiEnableCs

Description	This parameter enables or not the Chip Select handling functions. If True then Chip Select signal is generated and if False then the Chip Select signal would not be generated.
Class	Autosar Parameter
Range	True / False
Default	False
Source File	Spi_PbCfg.c
Source Representation	The bold section of the following structure: <pre> static CONST(Spi_LLD_DeviceAttributesConfig, SPI_CONST) DspiDeviceAttributesConfig_PB0[1] = </pre>

Table 3-79. SpiEnableCs

	<pre> { { (uint32)(DSPI_CTAR_CPOL_HIGH DSPI_CTAR_CPHA_LEADING DSPI_CTAR_PCSClk_3 DSPI_CTAR_SCSClk_64 DSPI_CTAR_PASC_3 DSPI_CTAR_SASC_128 DSPI_CTAR_PDT_3 DSPI_CTAR_SDT_64 DSPI_CTAR_PBR_5 DSPI_CTAR_SBR_16 DSPI_CTAR_DBR_0), uint32)(DSPI_CMD_PCS0 /* Chip Select Pin */ DSPI_CMD_CONTINUOUS_TRUE), /* Continuous chip select */ (uint32)DSPI_PCS0_IDLEHIGH /* Chip select polarity */ }, }; </pre>
--	---

Note

When SpiEnableCs is true, the corresponding chip select and SpiCsContinuous parameter values would be generated in the configuration and CS signal will be handled by SPI driver. If SpiEnableCs is false, the value generated by the configuration tool would be 0x00000000. When SpiEnableCs is disabled, the driver would not generate the CS at all, so in that case customer should take care of chip select handling in their software, and it will work only in case a sequence is configured to handle jobs on a unique device.

3.9.2.21 SpiShiftClockIdleLevel**Table 3-80. SpiShiftClockIdleLevel**

Description	This parameter defines the SPI shift clock idle level.
Class	Autosar Parameter
Range	High/ Low
Default	High
Source File	Spi_PbCfg.c
Source Representation	The bold section of the following structure: <pre> static CONST(Spi_LLD_DeviceAttributesConfig, SPI_CONST) DspiDeviceAttributesConfig_PB0[1] = { { (uint32)(DSPI_CTAR_CPOL_HIGH DSPI_CTAR_CPHA_LEADING DSPI_CTAR_PCSClk_3 DSPI_CTAR_SCSClk_64 DSPI_CTAR_PASC_3 DSPI_CTAR_SASC_128 DSPI_CTAR_PDT_3 DSPI_CTAR_SDT_64 DSPI_CTAR_PBR_5 DSPI_CTAR_SBR_16 DSPI_CTAR_DBR_0), (uint32)(DSPI_CMD_PCS0 /* Chip Select Pin */ DSPI_CMD_CONTINUOUS_TRUE), /* Continuous chip select */ (uint32)DSPI_PCS0_IDLEHIGH/* Chip select polarity */ }, }; </pre>

3.9.2.22 SpiTimeClk2Cs

Table 3-81. SpiTimeClk2Cs

Description	This parameter is the minimum time between clock and Chip Select.
Class	Autosar Parameter
Range	0 to 0.0001 (s)
Default	0.000001
Source File	Spi_PbCfg.c
Source Representation	The bold section of the following structure: <pre>static CONST(Spi_LLD_DeviceAttributesConfig, SPI_CONST) DspiDeviceAttributesConfig_PB0[1] = { { (uint32)(DSPI_CTAR_CPOL_HIGH DSPI_CTAR_CPHA_LEADING DSPI_CTAR_PCSClk_3 DSPI_CTAR_SCSClk_64 DSPI_CTAR_PASC_3 DSPI_CTAR_SASC_128 DSPI_CTAR_PDT_3 DSPI_CTAR_SDT_64 DSPI_CTAR_PBR_5 DSPI_CTAR_SBR_16 DSPI_CTAR_DBR_0), (uint32)(DSPI_CMD_PCS0 /* Chip Select Pin */ DSPI_CMD_CONTINOUS_TRUE), /* Continous chip select */ (uint32)DSPI_PCS0_IDLEHIGH /* Chip select polarity */ } };</pre>

3.9.2.23 SpiTimeCs2Clk

Table 3-82. SpiTimeCs2Clk

Description	This parameter is the time between Chip Select and next assertion of the clock
Class	Autosar Parameter
Range	0 to 0.01 (s)
Default	0.000001
Source File	Spi_PbCfg.c
Source Representation	The bold section of the following structure: <pre>static CONST(Spi_LLD_DeviceAttributesConfig, SPI_CONST) DspiDeviceAttributesConfig_PB0[1] = { { (uint32)(DSPI_CTAR_CPOL_HIGH DSPI_CTAR_CPHA_LEADING DSPI_CTAR_PCSClk_3 DSPI_CTAR_SCSClk_64 DSPI_CTAR_PASC_3 DSPI_CTAR_SASC_128 DSPI_CTAR_PDT_3 DSPI_CTAR_SDT_64 DSPI_CTAR_PBR_5 DSPI_CTAR_SBR_16 DSPI_CTAR_DBR_0),</pre>

Table 3-82. SpiTimeCs2Clk

	<pre> (uint32)(DSPI_CMD_PCS0 /* Chip Select Pin */ DSPI_CMD_CONTINUOUS_TRUE), /* Continuous chip select */ (uint32)DSPI_PCS0_IDLEHIGH /* Chip select polarity */ }, }; </pre>
--	---

3.9.2.24 SpiTimeCs2Cs

Table 3-83. SpiTimeCs2Cs

Description	This parameter defines the time between the Chip selects assertion in microseconds.
Class	Implementation specific parameter
Range	0 to 0.01(s)
Default	0.0000064
Source File	Spi_PbCfg.c
Source Representation	The bold section of the following structure: <pre> static CONST(Spi_LLD_DeviceAttributesConfig,SPI_CONST) DspiDeviceAttributesConfig_PB0[1] = { { (uint32)(DSPI_CTAR_CPOL_HIGH DSPI_CTAR_CPHA_LEADING DSPI_CTAR_PCSClk_3 DSPI_CTAR_SCSClk_64 DSPI_CTAR_PASC_3 DSPI_CTAR_SASC_128 DSPI_CTAR_PDT_3 DSPI_CTAR_SDT_64 DSPI_CTAR_PBR_5 DSPI_CTAR_SBR_16 DSPI_CTAR_DBR_0), (uint32)(DSPI_CMD_PCS0 /* Chip Select Pin */ DSPI_CMD_CONTINUOUS_TRUE), /* Continuous chip select */ (uint32)DSPI_PCS0_IDLEHIGH /* Chip select polarity */ }, }; </pre>

3.9.2.25 SpiCsContinuous

Table 3-84. SpiCsContinuous

Description	This parameter determines to keep chip selected between frame transfers.
Class	Implementation specific parameter
Range	True / false
Default	True
Source File	Spi_PbCfg.c
Source Representation	The bold section of the following structure: <pre> static CONST(Spi_LLD_DeviceAttributesConfig,SPI_CONST) </pre>

Table 3-84. SpiCsContinuous

	<pre> DspiDeviceAttributesConfig_PB0[1] = { { (uint32)(DSPI_CTAR_CPOL_HIGH DSPI_CTAR_CPHA_LEADING DSPI_CTAR_PCSCLK_3 DSPI_CTAR_SCSCCLK_64 DSPI_CTAR_PASC_3 DSPI_CTAR_SASC_128 DSPI_CTAR_PDT_3 DSPI_CTAR_SDT_64 DSPI_CTAR_PBR_5 DSPI_CTAR_SBR_16 DSPI_CTAR_DBR_0), (uint32)(DSPI_CMD_PCS0 /* Chip Select Pin */ DSPI_CMD_CONTINOUS_TRUE), /* Continuous chip select */ (uint32)DSPI_PCS0_IDLEHIGH /* Chip select polarity */ }, }; </pre>
--	---

3.9.2.26 SpiMaxJob

Table 3-85. SpiMaxJob

Description	This parameter contains the number of Jobs configured. It will be gathered by tools during the configuration stage.
Class	Autosar Parameter
Range	-
Default	-
Source File	-
Source Representation	The bold section of following structure: <pre> /* SpiDriverConfig_PB Configuration */ CONST(Spi_ConfigType, SPI_CONST) SpiDriverConfig_PB = { 1, 6, 5, 4, SpiChannelConfig_PB0, SpiJobConfig_PB0, SpiSequenceConfig_PB0, &SpiAttributesConfig_PB0, HWUnitConfig_PB }; </pre>

Note

SpiMaxJob parameter is not used, instead max jobs value is derived from number of jobs configured. SpiMaxJob parameter is retained in configuration for consistency with AUTOSAR_EcucParamDef.xml rev 003.

3.9.2.27 SpiHwUnit

Table 3-86. SpiHwUnit

Description	This parameter is the symbolic name to identify the HW SPI Hardware microcontroller peripheral allocated to this Job.
Class	Autosar Parameter
Range	CSIB0, CSIB1, CSIB2, CSIB3, CSIB4, CSIB5
Default	-
Source File	Spi_PbCfg.c
Source Representation	The bold section of following structure: <pre>static CONST(Spi_JobConfig, SPI_CONST) SpiJobConfig_PB0[5] = { { /* JOB_WREN_EEPROM1 */ (Spi_ChannelType)1u, JOB_WREN_EEPROM1_ChannelAssignment_PB, /* List of Channels */ EndJob_WREN_EEPROM1, /* End Notification */ (sint8)0, /* Priority */ &SpiJobState[0], /* JobState instance */ CSIB0, DSPI_0_OFFSET, /* DSPI device HW unit offset */ /* External Device Settings */ EEPROM3_SPI2_CS0 { ... } }, };</pre>

3.9.2.28 SpiJobId

Table 3-87. SpiJobId

Description	Job unique identifier in the configuration.
Class	Autosar Parameter
Range	0 – (SpiMaxJob-1)
Default	0
Source File	-
Source Representation	#define <JobName> ((Spi_JobType)<SpiJobId>)

3.9.2.29 SpiJobPriority

Table 3-88. SpiJobPriority

Description	This parameter defines the Job priority.
--------------------	--

Table continues on the next page...

Table 3-88. SpiJobPriority (continued)

Class	Autosar Parameter
Range	0 – 3
Default	0
Source File	-
Source Representation	The bold section of the following structure <pre> static CONST(Spi_JobConfig, SPI_CONST) SpiJobConfig_PB0[5] = { { /* JOB_WREN_EEPROM1 */ (Spi_ChannelType)1u, JOB_WREN_EEPROM1_ChannelAssignment_PB, /* List of Channels */ EndJob_WREN_EEPROM1, /* End Notification */ (sint8)0, /* Priority */ &SpiJobState[0], /* JobState instance */ CSIB0, DSPI_0_OFFSET, /* DSPI device HW unit offset */ /* External Device Settings */ EEPROM3_SPI2_CS0 } ... }; </pre>

3.9.2.30 SpiChannelAssignment

Table 3-89. SpiChannelAssignment

Description	A Job refers to several channels. This parameter forms a reference to several channels.
Class	Autosar Parameter
Range	NA
Default	-
Source File	Spi_PbCfg.c
Source Representation	The bold portion of the following structure: <pre> JOB_WREN_EEPROM1_ChannelAssignment_PB[1] = {CH_WREN_CMD_EEPROM1}; static CONST(Spi_JobConfig, SPI_CONST) SpiJobConfig_PB0[5] = { { /* JOB_WREN_EEPROM1 */ (Spi_ChannelType)1u, JOB_WREN_EEPROM1_ChannelAssignment_PB, /* List of Channels */ EndJob_WREN_EEPROM1, /* End Notification */ (sint8)0, /* Priority */ &SpiJobState[0], /* JobState instance */ CSIB0, DSPI_0_OFFSET, /* DSPI device HW unit offset */ /* External Device Settings */ EEPROM3_SPI2_CS0 } ... }; </pre>

Table 3-89. SpiChannelAssignment

	<pre> { ... }, } </pre>
--	-----------------------------

3.9.2.31 SpiDeviceAssignment

Table 3-90. SpiDeviceAssignment

Description	This parameter forms a reference to the external device used by the Job.
Class	Autosar Parameter
Range	NA
Default	-
Source File	Spi_PbCfg.c
Source Representation	The bold section of the following structure <pre> static CONST(Spi_JobConfig, SPI_CONST) SpiJobConfig_PB0[5] = { { /* JOB_WREN_EEPROM1 */ (Spi_ChannelType)lu, JOB_WREN_EEPROM1_ChannelAssignment_PB, /* List of Channels */ EndJob_WREN_EEPROM1, /* End Notification */ (sint8)0, /* Priority */ &SpiJobState[0], /* JobState instance */ CSIB0, DSPI_0_OFFSET, /* DSPI device HW unit offset */ /* External Device Settings */ EEPROM3_SPI2_CS0 { ... } }, }; </pre>

3.9.2.32 SpiMaxSequence

Table 3-91. SpiMaxSequence

Description	This parameter contains the number of Sequences configured. It will be gathered by tools during the configuration stage.
Class	Autosar Parameter
Range	-
Default	-
Source File	-
Source Representation	The bold section of following structure:

Table 3-91. SpiMaxSequence

	<pre> /* SpiDriverConfig_PB Configuration */ CONST(Spi_ConfigType, SPI_CONST) SpiDriverConfig_PB = { 1, 6, 5, 4, SpiChannelConfig_PB0, SpiJobConfig_PB0, SpiSequenceConfig_PB0, &SpiAttributesConfig_PB0, HWUnitConfig_PB }; </pre>
--	---

Note

SpiMaxSequence parameter is not used, instead max Sequences value is derived from number of Sequences configured.

SpiMaxSequence parameter is retained in configuration for consistency with AUTOSAR_EcucParamDef.xml rev 003.

3.9.2.33 SpiInterruptibleSequence**Table 3-92. SpiInterruptibleSequence**

Description	This parameter determines whether to allow or not this Sequence to be suspended by another one.
Class	Autosar Parameter
Range	True/False
Default	-
Source File	Spi_PbCfg.c
Source Representation	The bold section of the following structure: <pre> static CONST(Spi_SequenceConfig, SPI_CONST) SpiSequenceConfig_PB0[4] = { { (uint16)1u, SEQ_WREN_EEPROM1_JobAssignment_PB,/* List of Jobs */ Spi_Notification_Seq1,/* End Notification */ FALSE/* Interruptible */ }, }; </pre>

3.9.2.34 SpiSeqEndNotification**Table 3-93. SpiSeqEndNotification**

Description	This parameter is a reference to a notification function.
--------------------	---

Table continues on the next page...

Table 3-93. SpiSeqEndNotification (continued)

Class	Autosar Parameter
Range	-
Default	-
Source File	Spi_PbCfg.c
Source Representation	The bold section of the following structure: <pre> static CONST(Spi_SequenceConfig, SPI_CONST) SpiSequenceConfig_PB0[4] = { { (uint16)1u, SEQ_WREN_EEPROM1_JobAssignment_PB, /* List of Jobs */ Spi_Notification_Seq1, /* End Notification */ FALSE /* Interruptible */ }, }; </pre>

3.9.2.35 SpiSequenceld

Table 3-94. SpiSequenceld

Description	Sequence unique identifier in the configuration.
Class	Autosar Parameter
Range	0 – (SpiMaxSequence-1)
Default	0
Source File	-
Source Representation	#define <SequenceName> ((Spi_SequenceType)<SpiSequenceId>)

3.9.2.36 SpiJobAssignment

Table 3-95. SpiJobAssignment

Description	A sequence references several jobs, which are executed during a communication sequence. This parameter forms a reference to several Jobs.
Class	Autosar Parameter
Range	NA
Default	-
Source File	Spi_PbCfg.c
Source Representation	The bold section of the following structure <pre> static CONST(Spi_JobType, SPI_CONST) SEQ_WREN_EEPROM1_JobAssignment_PB[1] = {JOB_WREN_EEPROM1}; </pre>

Table 3-95. SpiJobAssignment

	<pre>static CONST(Spi_SequenceConfig, SPI_CONST) SpiSequenceConfig_PB0[4] = { { /* SEQ_WREN_EEPROM1 */ (uint16_t)lu, SEQ_WREN_EEPROM1_JobAssignment_PB, /* List of Jobs */ Spi_Notification_Seq1, /* End Notification */ FALSE /* Interruptible */ }, };</pre>
--	---

3.9.2.37 SpiJobEndNotification

Table 3-96. SpiJobEndNotification

Description	This parameter is a reference to a notification function. This function is editable only if SpiLevelDelivered is configured for either LEVEL1 or LEVEL2.
Class	Autosar Parameter
Range	-
Default	-
Source File	Spi_PbCfg.c
Source Representation	The bold section of the following structure: <pre>static CONST(Spi_JobConfig, SPI_CONST) SpiJobConfig_PB0[5] = { { /* JOB_WREN_EEPROM1 */ (Spi_ChannelType)lu, JOB_WREN_EEPROM1_ChannelAssignment_PB, /* List of Channels */ EndJob_WREN_EEPROM1, /* End Notification */ (uint8_t)0, /* Priority */ &SpiJobState[0], /* JobState instance */ CSIB0, DSPI_0_OFFSET, /* DSPI device HW unit offset */ /* External Device Settings */ EEPROM3_SPI2_CS0 }, };</pre>

3.9.2.38 SpiJobstartNotification

Table 3-97. SpijobstartNotification

Description	This parameter is a reference to a notification function.
--------------------	---

Table continues on the next page...

Table 3-97. SpijobstartNotification (continued)

Class	Non-Autosar Parameter
Range	NA
Default	-
Source File	Spi_PBCfg.c
Source Representation	The bold section of the following structure <pre> /* SpiJobConfig_PB0 Job Configuration of DRV_MCP*/ static CONST(Spi_JobConfig, SPI_CONST) SpiJobConfig_PB0[58] = { { /* JOB_DSPI1_EXP_P1_C67_N */ (Spi_ChannelType)1u, JOB_DSPI1_EXP_P1_C67_N_ChannelAssignment_PB, /* List of Channels */ EndJob_DSPI1_EXP_P1_C67, /* End Notification */ StartJob_DSPI1_EXP_C67, /* job Start Notification */ (sint8)1, /* Priority */ &SpiJobState[0], /* JobState instance */ CSIB1, /* HWUnit index */ DSPI_1_OFFSET, /* DSPI device HW unit offset */ /* External Device Settings */ DEV_EXP_100K_LEAD_0, /* External Device */ { (uint32)(DSPI_CTAR_CPOL_LOW /* Clock Polarity (Idle State) */ DSPI_CTAR_CPHA_LEADING /* Clock Phase */ DSPI_CTAR_PCCLK_3 DSPI_CTAR_SCCLK_512 /* TimeCs2Clk: Should=100.0, Is=96, Error=-4.0% */ DSPI_CTAR_PASC_3 DSPI_CTAR_SASC_256 /* TimeClk2Cs: Should=50.0, Is=48, Error=-4.0% */ DSPI_CTAR_PDT_3 DSPI_CTAR_SDT_512 /* TimeCs2Cs: Should=100.0, Is=96, Error=-4.0% */ DSPI_CTAR_PBR_5 DSPI_CTAR_SBR_32 DSPI_CTAR_DBR_0), /* Baudrate: Should=100000.0, Is=100000, Error=0.0% */ 0x00000000u, (uint32)0u /* Chip select polarity */ } }, }; </pre>

Chapter 4

How to use the same channel in multiple post build Configurations

The Same symbolic name is allowed for the channels which are common across multiple PB configurations by defining a common name if the channelid is same..

Scenario-1:

Let's assume there are 2 post build configurations, CFG1 and CFG2. CFG1 defines 3 channels, CFG2 defines 4 channels. Two of these channels are the same across configurations, let's call them COMCH1 and COMCH2.

So here are the channels defined by the 2 configurations: CFG1: COMCH1, COMCH2, OTHERCH1 CFG2: COMCH1, COMCH2, OTHERCH2, OTHERCH3

This means that CFG1 will use indexes 0, 1, 2, while CFG2 will use indexes 0,1,2,3.

The generated symbolic names for CFG1 and CFG2 will be:

```
#define COMCH1 0
```

```
#define COMCH2 1
```

```
#define OTHERCH1 2
```

```
#define OTHERCH2 2
```

```
#define OTHERCH3 3
```

It allows the upper layer to use same channel symbolic name(s) across multiple configurations.

Scenario-2:

Let's assume there are 4 post build configurations, CFG1, CFG2, CFG3 and CFG4. CFG1 defines 5 channels, CFG2 defines 1 channel, CFG3 defines 3 channels, CFG4 defines 4 channels. One of these channel is same across configurations, let's call it as COMCH1.

So here are the channels defined by the 2 configurations:

CFG1: OTHERCH1, COMCH1, OTHERCH2, OTHERCH3, OTHERCH4

CFG2: OTHERCH5

CFG3: OTHERCH6, COMCH1, OTHERCH7

CFG4: OTHERCH8, COMCH1, OTHERCH9, OTHERCH10

This means that CFG1 will use indexes 0, 1, 2,3,4,CFG2 will use indexes 0, CFG3 will use indexes 0,1,2 and CFG4 will use indexes 0,1,2,3.

The generated symbolic names will be:

```
#define OTHERCH1 0
```

```
#define COMCH1 1
```

```
#define OTHERCH2 2
```

```
#define OTHERCH3 3
```

```
#define OTHERCH4 4
```

```
#define OTHERCH5 0
```

```
#define OTHERCH6 0
```

```
#define OTHERCH7 2
```

```
#define OTHERCH8 0
```

```
#define OTHERCH9 2
```

```
#define OTHERCH10 3
```

Here the same symbolic name is generated for the channel "COMCH1" across the CFG1, CF3 and CFG4. It allows the upper layer to use same channel symbolic name(s) across multiple configurations.

Note: The same channelid should be used across multiple PB configurations in order to use same symbolic name across multiple PB configurations. All channel attributes shall be identical for cloned channels.,

How to Reach Us:**Home Page:**freescale.com**Web Support:**freescale.com/support

Information in this document is provided solely to enable system and software implementers to use Freescale products. There are no express or implied copyright licenses granted hereunder to design or fabricate any integrated circuits based on the information in this document.

Freescale reserves the right to make changes without further notice to any products herein. Freescale makes no warranty, representation, or guarantee regarding the suitability of its products for any particular purpose, nor does Freescale assume any liability arising out of the application or use of any product or circuit, and specifically disclaims any and all liability, including without limitation consequential or incidental damages. "Typical" parameters that may be provided in Freescale data sheets and/or specifications can and do vary in different applications, and actual performance may vary over time. All operating parameters, including "typicals," must be validated for each customer application by customer's technical experts. Freescale does not convey any license under its patent rights nor the rights of others. Freescale sells products pursuant to standard terms and conditions of sale, which can be found at the following address: freescale.com/SalesTermsandConditions.

Freescale, the Freescale logo, Altivec, C-5, CodeTest, CodeWarrior, ColdFire, ColdFire+, C-Ware, Energy Efficient Solutions logo, Kinetis, mobileGT, PowerQUICC, Processor Expert, QorIQ, Qorivva, StarCore, Symphony, and VortiQa are trademarks of Freescale Semiconductor, Inc., Reg. U.S. Pat. & Tm. Off. Airfast, BeeKit, BeeStack, CoreNet, Flexis, Layerscape, MagniV, MXC, Platform in a Package, QorIQ Qonverge, QUICC Engine, Ready Play, SafeAssure, SafeAssure logo, SMARTMOS, Tower, TurboLink, Vybrid, and Xtrinsic are trademarks of Freescale Semiconductor, Inc. All other product or service names are the property of their respective owners.

© 2014 Freescale Semiconductor, Inc.

Document Number UM13SPIASR4.0R1.0.1
Revision 1.2